



Near-optimal Algorithms for Stochastic Online Bin Packing

NIKHIL AYYADEVARA, Department of Computer Science, University of Michigan, Ann Arbor, Michigan, USA

RAJNI DABAS, Department of Computer Science, Northwestern University, Evanston, Illinois, USA
and Department of Computer Science, University of Delhi, New Delhi, India

ARINDAM KHAN and K. V. N. SREENIVAS, Department of Computer Science and Automation,
Indian Institute of Science, Bangalore, India

We study the online bin packing problem under two stochastic settings. In the bin packing problem, we are given n items with sizes in $(0, 1]$ and the goal is to pack them into the minimum number of unit-sized bins.

First, we study bin packing under the i.i.d. model, where item sizes are sampled independently and identically from a distribution in $(0, 1]$. Both the distribution and the total number of items are unknown. The items arrive one by one and their sizes are revealed upon their arrival and they must be packed immediately and irrevocably in bins of size 1. We provide a simple meta-algorithm that takes an offline α -asymptotic approximation algorithm and provides a polynomial-time $(\alpha + \varepsilon)$ -competitive algorithm for online bin packing under the i.i.d. model, where $\varepsilon > 0$ is a small constant. Using the AFPTAS for offline bin packing, we thus provide a linear time $(1 + \varepsilon)$ -competitive algorithm for online bin packing under i.i.d. model, thus settling the problem.

We then study the random-order model, where an adversary chooses the instance, but the order of arrival of items in the instance is drawn uniformly at random from the set of all permutations of the items. Kenyon's seminal result (1996) showed that the Best-Fit algorithm has a competitive ratio of at most $3/2$ in the random-order model, and conjectured the ratio to be ≈ 1.15 . However, it has been a long-standing open problem to break the barrier of $3/2$ even for special cases. Recently, Albers et al. (2021) showed an improvement by proving that in the special case when all the item sizes are greater than $1/3$, Best-Fit has a competitive ratio of at most $5/4$ in the random-order model. In this work, we settle this special case by showing that Best-Fit has a competitive ratio of exactly 1, i.e., Best-Fit performs almost optimally in this special case in the random-order model. We also make further progress by breaking the barrier of $3/2$ for the *3-Partition* problem, a notoriously hard special case of bin packing, where all item sizes lie in $(1/4, 1/2]$.

CCS Concepts: • **Theory of computation** → **Online algorithms**; *Stochastic approximation*; *Packing and covering problems*;

A preliminary version of this work appeared in the 49th EATCS International Colloquium on Automata, Languages and Programming (ICALP), 2022.

A. Khan's research is supported in part by Google India Research Award, SERB Core Research Grant (CRG/2022/001176) on "Optimization under Intractability and Uncertainty," and the Walmart Center for Tech Excellence at IISc (CSR Grant No. WMGT-23-0001). K. V. N. Sreenivas is grateful to the Google PhD Fellowship Program for supporting his research. A part of this work was done when N. Ayyadevara and R. Dabas were interns at the Indian Institute of Science Bengaluru.

Authors' Contact Information: Nikhil Ayyadevara, Computer Science, University of Michigan, Ann Arbor, Michigan, USA; e-mail: nikhil.ayyadevara@gmail.com; Rajni Dabas, Department of Computer Science, Northwestern University, Evanston, Illinois, USA and Department of Computer Science, University of Delhi, New Delhi, India; e-mail: rajni@cs.du.ac.in, rajni.dabas1@northwestern.edu; Arindam Khan, Department of Computer Science and Automation, Indian Institute of Science, Bangalore, India; e-mail: arindamkhan@iisc.ac.in; K. V. N. Sreenivas (corresponding author), Department of Computer Science and Automation, Indian Institute of Science, Bangalore, India; e-mail: venkatanaga@iisc.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1549-6333/2025/5-ART24

<https://doi.org/10.1145/3728642>

Additional Key Words and Phrases: Best-Fit Algorithm, Concentration Inequalities, IID Model, Online Bin Packing, Random-order Model, Upright Matching

ACM Reference format:

Nikhil Ayyadevara, Rajni Dabas, Arindam Khan, and K. V. N. Sreenivas. 2025. Near-optimal Algorithms for Stochastic Online Bin Packing. *ACM Trans. Algor.* 21, 2, Article 24 (May 2025), 39 pages.

<https://doi.org/10.1145/3728642>

1 Introduction

Bin packing (BP) is a fundamental NP-hard combinatorial optimization problem. In BP, we are given a set I of n items where the i th item has weight (also called size) $x_i \in (0, 1]$ and the goal is to partition I into the minimum number of sets (bins) such that the total weight of each set is at most 1. The problem has numerous applications in logistics, scheduling, cutting stock, and so on. [11]. Theoretically, BP has been the cornerstone for approximation and online algorithms and the study of the problem has led to the development of several interesting techniques [16, 33, 36].

Generally, the performance guarantee of an offline (resp. online) BP algorithm $\mathcal{A}$ is measured by **asymptotic approximation ratio (AAR)** (resp. **competitive ratio (CR)**). Let $\text{Opt}(I)$ and $\mathcal{A}(I)$ be the objective values returned by the optimal (offline) algorithm and algorithm $\mathcal{A}$, respectively, on an input I . Then AAR (resp. CR) is defined as

$$R_{\mathcal{A}}^{\infty} := \limsup_{m \rightarrow \infty} \left(\sup_{I: \text{Opt}(I)=m} \frac{\mathcal{A}(I)}{\text{Opt}(I)} \right).$$

Note that $R_{\mathcal{A}}^{\infty}$ focuses on instances where $\text{Opt}(I)$ is large and avoids pathological instances with large approximation ratios where $\text{Opt}(I)$ is small.

Best-Fit (BF), **First-Fit (FF)**, and **Next-Fit (NF)** are the three most commonly used algorithms for BP. Given x_i as the current item to be packed, they work as follows:

- BF: Pack x_i into the *fullest possible* bin; open a new bin if necessary.
- FF: Pack x_i into the *first possible* bin; open a new bin if necessary.
- NF: Pack x_i into the *most recently opened* bin; open a new bin if necessary.

Johnson et al. [30] studied several heuristics for BP such as BF, FF, **Best-Fit-Decreasing (BFD)**, **First-Fit-Decreasing (FFD)** and showed their (asymptotic) approximation guarantees to be $17/10$, $17/10$, $11/9$, $11/9$, respectively. Bekesi et al. [7] gave an $O(n)$ time $5/4$ -asymptotic approximation algorithm. Another $O(n \log n)$ time algorithm is **Modified-First-Fit-Decreasing (MFFD)** [31] which attains an AAR of $71/60 \approx 1.1834$. Vega and Lueker [16] gave an **asymptotic fully polynomial-time approximation scheme (AFPTAS)** for BP: For any $\varepsilon \in (0, 1/2)$, it returns a solution with at most $(1 + \varepsilon)\text{Opt}(I) + O(1)^1$ bins in time $C_{\varepsilon} + Cn \log 1/\varepsilon$, where C is an absolute constant and C_{ε} depends only on ε . Shortly after that, Karmarkar and Karp [33] gave an algorithm that returns a solution using $\text{Opt}(I) + O(\log^2 \text{Opt}(I))$ bins. The present best approximation is due to Hoberg and Rothvoss [27] which returns a solution using $\text{Opt}(I) + O(\log \text{Opt}(I))$ bins.

The 3-Partition problem is a notoriously hard special case of BP where all item sizes are larger than $1/4$. Eisenbrand et al. [18] mentioned that “much of the hardness of BP seems to appear already in the special case of 3-Partition when all item sizes are in $(1/4, 1/2]$.” This problem has deep connections with Beck’s conjecture in discrepancy theory [41, 48]. In fact, Hoberg and Rothvoss [27] conjectured that these 3-Partition instances are indeed the hardest instances for BP

¹In bin packing and related problems, the accuracy parameter ε is assumed to be a constant. Here, the term $O(1)$ hides some constants depending on ε .

and the additive integrality gap of the BP configuration LP for these 3-Partition instances is already $\Theta(\log n)$.

In online BP, items appear one by one and are required to be packed immediately and irrevocably. Lee and Lee [36] presented the Harmonic algorithm with CR $T_\infty \approx 1.691$, which is optimal for $O(1)$ space algorithms. For general online BP, the present best upper and lower bounds for the CR are 1.57829 [4] and 1.54278 [5], respectively.

In this article, we focus on online BP under a stochastic setting called the **identically distributed (i.i.d.)** model [12] where the input items are sampled from a sequence of independent and i.i.d. random variables. Here, the performance of an algorithm is measured by the **expected competitive ratio (ECR)**:

$$ER_{\mathcal{A}} := \lim_{n \rightarrow \infty} \frac{\mathbb{E}[\mathcal{A}(I_n(F))]}{\mathbb{E}[\text{Opt}(I_n(F))]},$$

where $I_n(F) := (X_1, X_2, \dots, X_n)$ is a list of n random variables drawn i.i.d. according to some unknown distribution F with support in $(0, 1]$. Mostly, BP has been studied under continuous uniform (denoted by $U[a, b]$, $0 \leq a < b \leq 1$, where item sizes are chosen uniformly from $[a, b]$) or discrete uniform distributions (denoted by $U\{j, k\}$, $1 \leq j \leq k$, where item sizes are chosen uniformly from $\{1/k, 2/k, \dots, j/k\}$). For $U[0, 1]$, Coffman et al. [14] showed that NF has an ECR of $4/3$ and Lee and Lee [37] showed that the Harmonic algorithm has an ECR of $\pi^2/3 - 2 \approx 1.2899$. Interestingly, Bentley et al. [8] showed that the ECR of FF as well as BF converges to 1 for $U[0, 1]$. It was later shown that the expected wasted space (i.e., the number of needed bins minus the total size of items) is $\Theta(n^{2/3})$ for FF [13, 47] and $\Theta(\sqrt{n} \log^{3/4} n)$ for BF [38, 47]. Rhee and Talagrand [46] exhibited an algorithm that, w.h.p., achieves a packing in $\text{Opt} + O(\sqrt{n} \log^{3/4} n)$ bins for any distribution F on $(0, 1]$. However, note that their CR can be quite bad when $\text{Opt} \ll n$. A distribution F is said to be *perfectly packable* if the expected wasted space in the optimal solution is $o(n)$ (i.e., nearly all bins in an optimal packing are almost fully packed). Csirik et al. [15] studied the **sum-of-squares (SS)** algorithm and showed that for any perfectly packable distribution, the expected wasted space is $O(\sqrt{n})$. However, for distributions that are not perfectly packable, the SS algorithm has an ECR of at most 3 and can have an ECR of $3/2$ in the worst-case [15]. For any discrete distribution, they gave an algorithm with an ECR of 1 that runs in pseudo-polynomial time in expectation. Gupta et al. [25] also obtained similar $o(n)$ expected wasted space guarantee by using an algorithm inspired by the interior-point (primal-dual) solution of the BP LP. However, it remains an open problem to obtain a polynomial-time $(1 + \varepsilon)$ -competitive algorithm for online BP under the i.i.d. model for arbitrary general distributions. In fact, the present best polynomial-time algorithm for BP under the i.i.d. model is BF which has an ECR of at most $3/2$. However, Albers et al. [1] showed that BF has an ECR ≥ 1.1 even for a simple distribution: when each item has size $1/4$ with probability $3/5$ and size $1/3$ with probability $2/5$.

We also study the *random-order model*, where the adversary specifies the items, but the arrival order is permuted uniformly at random. The performance measure in this model is called **asymptotic random order ratio (ARR)**:

$$RR_{\mathcal{A}}^\infty := \limsup_{m \rightarrow \infty} \left(\sup_{I: \text{Opt}(I)=m} \frac{\mathbb{E}[\mathcal{A}(I_\sigma)]}{\text{Opt}(I)} \right).$$

Here, σ is drawn uniformly at random from $\mathcal{S}_n$, the set of permutations of n elements, and $I_\sigma := (x_{\sigma(1)}, \dots, x_{\sigma(n)})$ is the permuted list. The random-order model generalizes the i.i.d. model [1], thus the lower bounds in the random-order model can be obtained from the i.i.d. model. Kenyon in her seminal paper [35] studied BF under random-order and showed that $1.08 \leq RR_{BF}^\infty \leq 3/2$. Kenyon conjectured “ RR_{BF}^∞ lies somewhere around 1.15.” The conjecture, if true, raises the possibility of a

better alternative practical offline algorithm: First shuffle the items randomly, then apply BF. This then beats the AAR of $71/60 \approx 1.18$ of the present best practical algorithm MFFD. The conjecture has received a lot of attention in the past two decades and, only recently, the analysis on the upper bound has been improved by [26], who showed that $RR_{BF}^\infty < 3/2 - 10^{-10}$. They also showed an improved lower bound of 1.144 using a computer program to solve a large Markov chain. Coffman et al. [32] showed that $RR_{NF}^\infty = 2$. Fischer and Röglin [22] achieved analogous results for Worst-Fit [29] and Smart-NF [42]. Recently, Fischer [10] presented an exponential-time algorithm, claiming an ARR of $(1 + \epsilon)$.

Several other problems have been studied under the i.i.d. model and the random-order model [2, 17, 19–21, 23–25, 39].

Monotonicity is a natural property of BP algorithms, which holds if the algorithm never uses fewer bins to pack $\hat{I}$ when compared I , where $\hat{I}$ is obtained from I by increasing the item sizes. Murgolo [40] showed that while NF is monotone, BF and FF are not.

1.1 Our Contributions

BP under the i.i.d. Model. We achieve a near-optimal performance guarantee for the BP problem under the i.i.d. model, thus settling the problem. For any arbitrary unknown distribution F on $(0, 1]$, we give a meta-algorithm (see Section 3) that takes an α -asymptotic approximation algorithm as input and provides a polynomial-time $(\alpha + \epsilon)$ -competitive algorithm. Note that both the distribution F as well as the number of items n are unknown in this case. We also remark that the distribution F can depend on n .

THEOREM 1.1. *Let $\epsilon \in (0, 1)$ be a constant parameter. For online BP under the i.i.d. model, where n items are sampled from an unknown distribution F , given an offline algorithm $\mathcal{A}_\alpha$ with an AAR of α and runtime $\beta(n)$, there exists a meta-algorithm which returns a solution with an ECR of $(\alpha + \epsilon)$ and runtime $O(\beta(n))$.²*

Using an AFPTAS for BP (e.g., [16]) as $\mathcal{A}_\alpha$, we obtain the following corollary.

COROLLARY 1.2. *Using an AFPTAS for BP as $\mathcal{A}_\alpha$ in Theorem 1.1, we obtain an algorithm for online BP under the i.i.d. model with an ECR of $(1 + \epsilon)$ for any $\epsilon \in (0, 1/2)$.*

Most algorithms for BP under the i.i.d. model are based on the following idea. Consider a sequence of $2k$ items where each item is independently drawn from an unknown distribution F , and let $\mathcal{A}$ be a packing algorithm. Pack the first k items using $\mathcal{A}$; denote the packing by $\mathcal{P}'$. Similarly, let $\mathcal{P}''$ be the packing of the next k items using $\mathcal{A}$. Since each item is drawn independently from F , both $\mathcal{P}'$ and $\mathcal{P}''$ have the same properties in expectation; in particular, the expected number of bins used in $\mathcal{P}'$ and $\mathcal{P}''$ is the same. Thus, intuitively, we want to use the packing $\mathcal{P}'$ as a proxy for the packing $\mathcal{P}''$. However, there are two problems. First, we do not know n , which means that there is no way to know what a good sample size is. Second, we need to show the stronger statement that w.h.p. $\mathcal{P}' \approx \mathcal{P}''$. Note that the items in $\mathcal{P}'$ and $\mathcal{P}''$ are expected to be similar, but they may not be the same. So, it is not clear which item in $\mathcal{P}'$ is to be used as a proxy for a newly arrived item in the second half. Due to the online nature, erroneous choice of proxy items can be quite costly. Different algorithms handle this problem in different ways. Some algorithms exploit the properties of particular distributions, some use exponential or pseudo-polynomial time, and so on.

Rhee and Talagrand [45, 46] used *upright matching* to decide which item can be considered as a proxy for a newly arrived item.

²As mentioned in an earlier footnote, here the $O(\cdot)$ notation hides some constants depending on ϵ .

Table 1. Analysis of Our Meta-Algorithm Depending on $\mathcal{A}_\alpha$

$\mathcal{A}_\alpha$	Time Complexity	ECR
AFPTAS [16]	$O(C_\epsilon + Cn \log 1/\epsilon)$	$(1 + \epsilon)$
MFFD [31]	$O(n \log n)$	$(71/60 + \epsilon)$
BFD [28]	$O(n \log n)$	$(11/9 + \epsilon)$
FFD [28]	$O(n \log n)$	$(11/9 + \epsilon)$
NF-Decreasing [3]	$O(n \log n)$	$(T_\infty + \epsilon)$
Harmonic [36]	$O(n)$	$(T_\infty + \epsilon)$
NF	$O(n)$	$(2 + \epsilon)$

In the first row, C is an absolute constant and C_ϵ is a constant that depends on ϵ .

They consider the model packing $\mathcal{P}_k$ of the first k items (let's call these the proxy items) using an offline algorithm. With the arrival of each of the next k items, they take a proxy item at random and pack it according to the model packing. Then, they try to fit in the real item using upright matching. They repeat this process until the last item is packed. However, they could only show a guarantee of $\text{Opt} + O(\sqrt{n} \log^{3/4} n)$. The main drawback of [46] is that their ECR can be quite bad if $\text{Opt} \ll n$ (say, $\text{Opt} = n^{2/3}$). One of the reasons for this drawback is that they don't distinguish between small and large items; when there are too many small items, the ECR blows up.

Using a similar approach, Fischer [10] obtained a $(1 + \epsilon)$ -competitive randomized algorithm for the random-order model, but it takes exponential time, and the analysis is quite complicated. The exponential time was crucial in finding the optimal packing which was then used as a good proxy packing. However, prior to our work, no polynomial-time algorithm existed which achieves a $(1 + \epsilon)$ CR.

To circumvent these issues, we treat large and small items separately. However, a straightforward adaptation faces several technical obstacles. Thus our analysis required intricate applications of concentration inequalities and sophisticated use of upright matching. First, we consider the semi-random case when we know n . Our algorithm works in stages. For a small constant $\delta \in (0, 1]$, the first stage contains only $\delta^2 n$ items. These items give us an estimate of the distribution. But since the first stage contains a very small fraction (δ^2) of the entire input, we use a simple algorithm by NF to pack it. If the packing (of the first stage) does not contain too many large items, we show that the simple NF algorithm suffices for the entire input. Otherwise, we use a proxy packing of the set of first $\delta^2 n$ items to pack the next $\delta^2 n$ items. In the process, the small and large items are packed in a different manner. The third set of $\delta^2 n$ number of items are packed using the proxy packing of the second set of $\delta^2 n$ number of items. This process continues until all the items arrive.

Finally, we get rid of the assumption that we know n by first guessing the value of n and then refining our guess if it is incorrect. First, we guess the value of n to be a constant n_0 . If it is incorrect, we increase our guess by multiplying n_0 with a small factor greater than 1. We continue this process of improving our guess until all the items arrive.

Our algorithm is simple, polynomial-time (in fact, $O(n)$ time), and achieves essentially the best possible CR. It is relatively simpler to analyze when compared to Fischer's algorithm [10]. Also, unlike the algorithms of Rhee and Talagrand [46] as well as Fischer [10], our algorithm is deterministic. This is because, unlike their algorithms, instead of taking proxy items at random, we pack all the proxy items before the start of a stage and try to fit in the real items as they come. This makes our algorithm deterministic. Our algorithm is explained in detail in Section 3.2. The nature of the meta-algorithm provides flexibility and ease of application. See Table 1 for the performance guarantees obtained using different offline algorithms.

See Section 3 for the details of the proof and the description of our algorithm.

BP under the Random-Order Model. Next, we study BP under the random-order model. Recently, Albers et al. [1] showed that BF is monotone if all the item sizes are greater than $1/3$. Using this result, they showed that in this special case, BF has an ARR of at most $5/4$. We show that, somewhat surprisingly, in this case, BF actually has an ARR of 1 (see Section 4.1 for the detailed proof).

THEOREM 1.3. *For online BP under the random-order model, BF achieves an asymptotic random-order ratio of 1 when all the item sizes are in $(1/3, 1]$.*

Next, we study the 3-partition problem, a special case of BP when all the item sizes are in $(1/4, 1/2]$. This is known to be an extremely hard case [27]. Albers et al. [1] mentioned that “it is sufficient to have one item in $(1/4, 1/3]$ to force BF into anomalous behavior.” For example, BF is non-monotone in the presence of items of size less than $1/3$. Thus the techniques of [1] do not extend to the 3-Partition problem. We break the barrier of $3/2$ in this special case, by showing that BF attains an ARR of at most 1.49107.

THEOREM 1.4. *For online BP under the random-order model, BF has an asymptotic random-order ratio of at most 1.49107 when all the item sizes are in $(1/4, 1/2]$.*

We prove Theorem 1.4 in Section 4.2. As 3-partition instances are believed to be the hardest instances for BP, our result gives a strong indication that the ARR of BF might be strictly less than $3/2$.

2 Notations and Preliminaries

Let I denote a list of n items where each item x has an associated weight given by $\mathcal{W}(x)$. We overload this notation and define $\mathcal{W}(I)$ as the sum of weights of all the items in I . For any $n \in \mathbb{N}_+$ we denote the set $\{1, 2, \dots, n\}$ by $[n]$. Let $\sigma : [n] \mapsto [n]$ denote a permutation of $[n]$. Then I_σ denotes the list I permuted according to σ , i.e., if x_i denotes the i th item in I ($i \in [n]$), then the i th item in I_σ is given by $x_{\sigma(i)}$.

2.1 The Upright Matching Problem

The upright matching problem was first introduced in [34] in the context of designing BP algorithms when the items are sampled from the uniform distribution. In upright matching, a set of m plus (+) points and a set of m minus (−) points are located in $\mathbb{R}^2$. Based on the configuration of the plus and minus points, we can create a graph $\mathcal{G}$ as follows. The vertices of $\mathcal{G}$ are the plus and minus points. There exists an edge between two vertices (p^+, p^-) iff

- The point p^+ is a plus point and the point p^- is a minus point.
- If $p^+ =: (x^+, y^+)$ and $p^- =: (x^-, y^-)$, then $x^+ \geq x^-$ and $y^+ \geq y^-$, i.e., the plus point p^+ lies to the *upright* of the minus point p^- .

The objective of the upright matching problem is to find the maximum matching in the defined graph $\mathcal{G}$. In other words, the objective is to minimize the number of unmatched points. Suppose P^+ denotes the set of plus points and P^- denotes the set of minus points. We denote the minimum possible number of unmatched points by $U(P^+, P^-)$.

It can be easily shown that for a given set of plus points P^+ and a set of minus points P^- , the minimum possible number of unmatched points can be found in the following way: We process the points in the increasing order of their x -coordinates. If a plus point P^+ arrives, we check if there exists an unmatched minus point that has already arrived and whose y -coordinate is less than that of P^+ . If no such minus point exists, then P^+ is left unmatched for the rest of the process. Otherwise, we match it with an already arrived, yet, unmatched minus point P^- with the maximum

Algorithm 1: Maximum Upright Matching

Input: A set P^+ of m plus points and a set P^- of m minus points.

- 1: Arrange $P^+ \cup P^-$ in increasing order of their x -coordinates to obtain a sequence P .
- 2: Initialize the set of unmatched minus points $U^- = \emptyset$.
- 3: **for** a point $p = (x, y)$ in P **do**
- 4: **if** $p \in P^-$, i.e., p is a minus point **then**
- 5: $U^- \leftarrow U^- \cup \{p\}$
- 6: **else if** $p \in P^+$, i.e., p is a plus point **then**
- 7: $V^- \leftarrow \{(x^-, y^-) \in U^- : y^- \leq y\}$
- 8: **if** $V^- = \emptyset$ **then**
- 9: Leave p to be unmatched
- 10: **else**
- 11: Define $p^- = \arg \max_{(x^-, y^-) \in V^-} y^-$
- 12: Match p^- and p
- 13: $U^- \leftarrow U^- \setminus \{p^-\}$
- 14: **end if**
- 15: **end if**
- 16: **end for**

y -coordinate among those whose y -coordinates are at most that of P^+ . A more formal pseudocode of this procedure is given in Algorithm 1.

It can be checked that Algorithm 1 correctly computes a maximum upright matching. A high-level proof can be found in [34]. However, for the sake of completeness, we provide a detailed proof in Appendix A.

2.1.1 Stochastic Variants of Upright Matching. We now discuss two stochastic variants of the upright matching problem, which, in one form or the another, have been used in various works on BP; e.g., [10, 34, 46, 47].

LEMMA 2.1 (UPRIGHT MATCHING WITH I.I.D. COORDINATES [10]). *Consider a set of real numbers $r_1, r_2, \dots, r_m, s_1, s_2, \dots, s_m$ which are independently and identically sampled from a distribution. (We remark that the distribution can be parameterized by m .) Define the set of minus points $P^- = \{(-1, r_i)\}_{i \in [m]}$, and the set of plus points $P^+ = \{(+1, s_i)\}_{i \in [m]}$. Then, there exist universal constants a, C, K such that with probability at least $1 - C \exp(-a(\log m)^{3/2})$, we have that*

$$U(P^+, P^-) \leq K\sqrt{m}(\log m)^{3/4}.$$

LEMMA 2.2 (UPRIGHT MATCHING WITH RANDOMLY PERMUTED COORDINATES [10]). *Consider a set of reals $r_1, r_2, \dots, r_m$ and $s_1, s_2, \dots, s_m$ such that $r_i \leq s_i$ for all $i \in [m]$. Consider a uniform random permutation π of $[2m]$. Define the set of minus points $P^- = \{(\pi(i), r_i)\}_{i \in [m]}$, and the set of plus points $P^+ = \{(\pi(m+i), s_i)\}_{i \in [m]}$. Then, there exist universal constants a, C, K such that with probability at least $1 - C \exp(-a(\log m)^{3/2})$, we have that*

$$U(P^+, P^-) \leq K\sqrt{m}(\log m)^{3/4}.$$

3 Online BP Problem under the i.i.d. Model

In this section, we provide the meta algorithm as described in Theorem 1.1. For the ease of presentation, we split the section into three subsections. In Section 3.1, we describe “Blueprint Packing,” which is one of the central ideas of the article. In Section 3.2, we assume a semi-online model, i.e., we

assume that the number of items n is known beforehand, and design an algorithm. The idea of blueprint packing is used extensively as a subroutine in designing this algorithm. Later, in Section 3.3, we get rid of the assumption on the knowledge of the number of items using a *doubling trick*.

Let the underlying distribution be F . Without loss of generality, we assume that the support set of F is a subset of $(0, 1]$. For any set of items J , we define $\mathcal{W}(J)$ as the sum of weights of all the items in J . For any $k \in \mathbb{N}_+$, we denote the set $\{1, 2, \dots, k\}$ by $[k]$. Let $\mathcal{A}_\alpha$ be an offline algorithm for BP with an AAR of $\alpha > 1$ and let Opt denote the optimal algorithm. Let $\varepsilon \in (0, 1)$ be a constant parameter and let $0 < \delta < \varepsilon/8$ be a constant such that $1/\delta$ is an integer. For any item x , we call x to be a *large* item if $x \geq \delta$ and a *small* item otherwise.

3.1 Blueprint Packing

One of the central ideas of our algorithm for online BP under the i.i.d. model is *blueprint packing*. Informally, it shows us how to pack a set of k i.i.d. items arriving online using the knowledge of a set of k items that are already present and are sampled from the same distribution independently.

Consider a set J_1 of k items ($k \in \mathbb{N}$) sampled independently from an arbitrary distribution. Consider an offline BP algorithm $\mathcal{A}_\alpha$ that has an AAR of α , and suppose we have the packing $\mathcal{A}_\alpha(J_1)$ at our disposal. Now, suppose that another set J_2 of k items sampled independently from the same distribution arrive online. We would like to pack the set J_2 (online) using the packing $\mathcal{A}_\alpha(J_1)$ as a blueprint.

LEMMA 3.1 (BLUEPRINT PACKING LEMMA). *Consider any offline BP algorithm $\mathcal{A}_\alpha$ with an AAR of α . Let $\mathcal{F}$ be any arbitrary distribution with its support in $(0, 1]$ and let J_1 be a set of k items sampled from $\mathcal{F}$ independently. Consider another set J_2 of k items sampled independently from $\mathcal{F}$ and let $\delta > 0$ be a small constant. Then, there exists an online algorithm that packs J_2 in at most*

$$\alpha(1 + 4\delta)\mathbb{E}[\text{Opt}(J_2)] + o(\mathbb{E}[\text{Opt}(J_2)])$$

number of bins, with high probability.

We next describe the blueprint packing procedure and prove Lemma 3.1. To pack the set J_2 , we first compute the packing $\mathcal{A}_\alpha(J_1)$ and refer to it as the *blueprint packing*. We refer to the large items in the blueprint packing $\mathcal{A}_\alpha(J_1)$ as *proxy items*. First, we remove all the small items in the blueprint packing $\mathcal{A}_\alpha(J_1)$. Then, in each bin of the modified packing $\mathcal{A}_\alpha(J_1)$, the empty space is designated to be an *S-slot*. The *S-slots* will be used to pack the small items of J_2 . (These *S-slots* can be thought of as empty bins of varying sizes to pack the small items of J_2 .) When a small item s of J_2 arrives, we pack it in these *S-slots* using NF. If s does not fit according to the NF rule, then we open a new bin and designate the entire bin as an *S-slot*, and pack s in there. On the other hand, if a large item ℓ in J_2 arrives, we remove the smallest proxy item in the blueprint packing $\mathcal{A}_\alpha(J_1)$ that is bigger than ℓ , if one exists. If no such proxy item exists, we open a new bin for ℓ , pack it there, and close the bin, i.e., this bin will not be used for any future items. (As a side note, we can possibly use this bin more efficiently, but we will see that it will not affect the performance ratio.) A pseudocode for the blueprint packing procedure can be found in Algorithm 2.

3.1.1 Concentration Bounds to Analyze Blueprint Packing. We will need some concentration inequalities and tail bounds to analyze the blueprint packing procedure. First, we state the Bernstein's inequality, which will be used heavily throughout. The details and proof can be found in [9].

LEMMA 3.2 (BERNSTEIN'S INEQUALITY). *Let $X_1, X_2, \dots, X_n$ be independent random variables such that each $X_i \in [0, 1]$. Then, for any $\lambda > 0$, the following inequality holds:*

$$\mathbb{P}\left[\left|\sum_{i=1}^n X_i - \sum_{i=1}^n \mathbb{E}[X_i]\right| \geq \lambda\right] \leq 2 \exp\left(-\frac{\lambda^2}{2(\sum_{i=1}^n \mathbb{E}[X_i] + \lambda/3)}\right).$$

Algorithm 2: Blueprint Packing Procedure $\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1)$: Pack the Set J_2 Online Using the Packing $\mathcal{A}_\alpha(J_1)$ as a Blueprint

Input: Two disjoint sets of items J_1 and J_2 where each item in $J_1 \cup J_2$ is sampled in an i.i.d. manner.

Objective: Pack the set J_2 online using the packing of the set J_1 by an offline algorithm $\mathcal{A}_\alpha$.

```

1: Construct the blueprint packing  $\mathcal{A}_\alpha(J_1)$ .
2: Initialize the set of  $S$ -slots  $S = \emptyset$ .
3: for each bin  $B$  in  $\mathcal{A}_\alpha(J_1)$  do
4:   Remove the small items in  $B$ .
5:   Create an  $S$ -slot  $H$  of size equal to  $(1 - \text{weight of all the large items in } B)$ .
6:    $S \leftarrow S \cup \{H\}$ .
7: end for
8: Initialize the set of proxy items  $D$  to the set of large items in the set  $J_1$ . ▷ The set of proxy items.
9: for an item  $x \in J_2$  do
10:  if  $x$  is large then
11:    if there exists a proxy item  $d \in D$  such that  $d \geq x$  then
12:      Find smallest such  $d$ .
13:       $D \leftarrow D \setminus \{d\}$ .
14:      Pack  $x$  in place of  $d$  in the packing  $\mathcal{A}_\alpha(J_1)$ .
15:    else
16:      Open a new bin and pack  $x$  and close the bin.
17:    end if
18:  else (i.e.,  $x$  is small)
19:    Try packing  $x$  in the set of  $S$ -slots  $S$  using Next-Fit.
20:    if  $x$  cannot be packed then
21:      Open a new bin  $B$  with a single  $S$ -slot of unit capacity.
22:       $S \leftarrow S \cup \{B\}$ .
23:      Pack  $x$  in  $B$ .
24:    end if
25:  end if
26: end for
27: return the packing after removing the proxy items that have not been replaced.

```

The following lemma is a direct implication of the results of [43, 46].

LEMMA 3.3. *For any $t \in [n]$, let $I(1, t)$ denote the first t items of a set I of n items sampled independently from a distribution. Then there exist constants $K, a > 0$ such that,*

$$\mathbb{P} \left[\text{Opt}(I(1, t)) \geq \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + K \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a (\log n)^{3/2} \right).$$

PROOF SKETCH. The following claim is similar to Theorem 2.1 in [46].

CLAIM 1. *Let I be a list of n items sampled independently from a distribution and let $I(1, t)$ denote the first t items. Then there exist constants $K_1, a_1 > 0$ such that*

$$\mathbb{P} \left[\text{Opt}(I(1, t)) \geq \frac{t}{n} \text{Opt}(I) + K_1 \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a_1 (\log n)^{3/2} \right).$$

The proof is almost the same as the proof of Theorem 2.1 in [46] with some small changes. However, since these changes are not very trivial, we give the full proof in Appendix B.

The next claim is a direct implication of the main result of [43].

CLAIM 2. *Let I be a list of n items sampled independently from a distribution. Then there exist constants $K_2, a_2 > 0$ such that*

$$\mathbb{P} \left[\mathbb{E} [\text{Opt}(I)] \geq \text{Opt}(I) + K_2 \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a_2 (\log n)^{3/2} \right).$$

Combining both claims, we obtain that there exist constants $K, a > 0$ such that

$$\mathbb{P} \left[\text{Opt}(I(1, t)) \geq \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + K \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a (\log n)^{3/2} \right).$$

The following lemmas are about how a property of a part of the input (say, the total size) compares to that of the entire input.

LEMMA 3.4. *For an input set I of n items drawn independently from a distribution, for any arbitrary set $J \subseteq I$ we have*

$$\mathbb{P} \left[\left| \mathcal{W}(J) - \frac{|J|}{n} \mathbb{E} [\mathcal{W}(I)] \right| \geq \mathbb{E} [\mathcal{W}(I)]^{2/3} \right] \leq 2 \exp \left(-\frac{1}{3} \mathbb{E} [\mathcal{W}(I)]^{1/3} \right).$$

PROOF. We can assume that $\mathcal{W}(I)$ goes to infinity since we know that $\text{Opt}(I) \leq 2\mathcal{W}(I) + 1$. Let $K := \mathbb{E} [\mathcal{W}(I)]$ and $\mathcal{W}(x)$ be the weight of item x . Using Bernstein's inequality (Lemma 3.2),

$$\begin{aligned} & \mathbb{P} \left[\left| \sum_{x \in J} \mathcal{W}(x) - \sum_{x \in J} \mathbb{E} [\mathcal{W}(x)] \right| \geq K^{2/3} \right] \\ & \leq 2 \exp \left(-\frac{K^{4/3}}{2 \left(\sum_{x \in J} \mathbb{E} [\mathcal{W}(x)] + K^{2/3}/3 \right)} \right) \\ & \leq 2 \exp \left(-\frac{K^{4/3}}{2 \left(K + K^{2/3}/3 \right)} \right) \\ & \leq 2 \exp \left(-\frac{K^{4/3}}{2 \left(K + K/3 \right)} \right) \quad (\text{since } K \text{ goes to infinity, } K^{2/3} \leq K) \\ & \leq 2 \exp \left(-\frac{1}{3} K^{1/3} \right). \end{aligned}$$

Since $\sum_{x \in J} \mathcal{W}(x) = \mathcal{W}(J)$ and since $\mathbb{E} [\mathcal{W}(J)] = \frac{|J|}{n} \mathbb{E} [\mathcal{W}(I)]$, the lemma follows. $\square$

LEMMA 3.5. *Let I be an input set of n items drawn independently from a distribution and let J be any subset of I . Suppose J_ℓ (resp. I_ℓ) denote the set of large items in J (resp. I). Then we have*

$$\mathbb{P} \left[\left| |J_\ell| - \frac{|J|}{n} \mathbb{E} [|I_\ell|] \right| \geq \mathbb{E} [\mathcal{W}(I)]^{2/3} \right] \leq 2 \exp \left(-\frac{\delta}{3} \mathbb{E} [\mathcal{W}(I)]^{1/3} \right).$$

PROOF. We can assume that $\mathcal{W}(I)$ goes to infinity. Let $K := \mathbb{E} [\mathcal{W}(I)]$. For any item x , let L_x be the indicator random variable which denotes if the item x is a large item or not. Using Bernstein's

inequality (Lemma 3.2),

$$\begin{aligned}
& \mathbb{P} \left[\left| \sum_{x \in J} L_x - \sum_{x \in J} \mathbb{E}[L_x] \right| \geq K^{2/3} \right] \\
& \leq 2 \exp \left(- \frac{K^{4/3}}{2 (\sum_{x \in J} \mathbb{E}[L_x] + K^{2/3}/3)} \right) \\
& = 2 \exp \left(- \frac{K^{4/3}}{2 (\mathbb{E}[|J_\ell|] + K^{2/3}/3)} \right) \quad (\text{since } \sum_{x \in J} L_x = |J_\ell|) \\
& \leq 2 \exp \left(- \frac{K^{4/3}}{2 (K/\delta + K^{2/3}/3)} \right) \quad (\text{since } K = \mathbb{E}[\mathcal{W}(I)] \geq \delta \mathbb{E}[|J_\ell|]) \\
& \leq 2 \exp \left(- \frac{\delta}{3} K^{1/3} \right).
\end{aligned}$$

Since $\sum_{x \in J} L_x = |J_\ell|$ and since $\mathbb{E}[|J_\ell|] = \frac{|J|}{n} \mathbb{E}[|I_\ell|]$, the lemma follows. $\square$

The next lemma shows that the optimal number of bins required to pack a set of items sampled independently from a distribution is approximately proportional to the number of items.

LEMMA 3.6. *For any $t \in \{1, 2, \dots, n\}$, let $I(1, t)$ denote the first t items of a set I of items sampled independently from a distribution. Then there exist constants $C, a > 0$ such that with probability at least $1 - \exp(-a (\log \mathbb{E}[\text{Opt}(I)])^{1/3})$, we have*

$$\text{Opt}(I(1, t)) \leq (1 + 2\delta) \frac{t}{n} \mathbb{E}[\text{Opt}(I)] + C \mathbb{E}[\text{Opt}(I)]^{2/3}.$$

PROOF. Let us denote the set of large items in $I(1, t)$ by $I_\ell(1, t)$ and denote the set of small items by $I_s(1, t)$. Consider any optimal packing of $I_\ell(1, t)$. Start packing $I_s(1, t)$ in the spaces left in the bins greedily using NF while opening new bins whenever necessary. This gives us a valid packing of $I(1, t)$.

We distinguish between the cases when we open new bins to pack $I_s(1, t)$ and when we do not. We will also use Lemmas 3.3 to 3.5 to bound the number of opened bins. Using a union bound, we can assume that there exists a constant a' such that the guarantees given by Lemmas 3.3 to 3.5 hold with probability at least $1 - \exp(-a' \mathbb{E}[\text{Opt}(I)]^{2/3})$. Define the event

$$E := \text{"We do not open any new bins for } I_s(1, t)\text{"}$$

Suppose the event E occurs with probability at least $\exp(-\frac{1}{2} a' (\log \mathbb{E}[\text{Opt}(I)])^{3/2})$. Note that since Lemma 3.3 (unconditionally) holds with probability at least $1 - \exp(-\frac{1}{2} a' (\log \mathbb{E}[\text{Opt}(I)])^{3/2})$, when we condition on the event E , it holds with probability at least $1 - \frac{\exp(-a' (\log \mathbb{E}[\text{Opt}(I)])^{3/2})}{\exp(-\frac{1}{2} a' (\log \mathbb{E}[\text{Opt}(I)])^{3/2})}$ which is equal to $1 - \exp(-\frac{1}{2} a' (\log \mathbb{E}[\text{Opt}(I)])^{3/2})$.

Now, we condition on event E . In this case, we have

$$\text{Opt}(I(1, t)) = \text{Opt}(I_\ell(1, t)). \quad (3.1)$$

We further consider two cases.

Case 1: $\mathbb{E}[|I_\ell|] \leq 2 \mathbb{E}[\mathcal{W}(I)]^{2/3}$.

Then we have

$$\begin{aligned}
 \text{Opt}(I(1, t)) &= \text{Opt}(I_\ell(1, t)) \\
 &\leq |I_\ell| \\
 &\leq \mathbb{E}[|I_\ell|] + \mathbb{E}[\mathcal{W}(I)]^{2/3} && \text{(using Lemma 3.5)} \\
 &\leq 3\mathbb{E}[\mathcal{W}(I)]^{2/3} \\
 &\leq 3\mathbb{E}[\text{Opt}(I)]^{2/3}.
 \end{aligned}$$

Now, we consider the other case.

Case 2: $\mathbb{E}[|I_\ell|] > 2\mathbb{E}[\mathcal{W}(I)]^{2/3}$.

Now, to bound $\text{Opt}(I_\ell(1, t))$, we can apply Lemma 3.3 confined to only large items, by treating other items as having zero weight. More formally, consider I and remove all the small items in I to obtain I_ℓ . Then, an item of I_ℓ can be thought of as sampling from a different distribution F_ℓ as follows. We sample an item from F ; if it is a small item, we discard it and sample until we get a large item.

Define

$$t' = \frac{t}{n}\mathbb{E}[|I_\ell|] + \mathbb{E}[\mathcal{W}(I)]^{2/3} \quad \text{and} \quad n' = \mathbb{E}[|I_\ell|] - \mathbb{E}[\mathcal{W}(I)]^{2/3}.$$

Owing to Lemma 3.5, we obtain that, with high probability,

$$|I_\ell(1, t)| \leq t', \tag{3.2}$$

$$|I_\ell| \geq n'. \tag{3.3}$$

Define I'_ℓ as the prefix of I_ℓ with n' items. Note that t' can be larger than n' . (This happens when, e.g., $t = n$.) Therefore, we consider two further subcases.

Case 2.1. $t' \leq n'$.

In this case, define

$$t'' := \frac{t}{n}n' = \frac{t}{n}\mathbb{E}[|I_\ell|] - \frac{t}{n}\mathbb{E}[\mathcal{W}(I)]^{2/3},$$

and note that

$$t' \leq t'' + 2\mathbb{E}[\mathcal{W}(I)]^{2/3}. \tag{3.4}$$

Define $I'_\ell(t')$ as the prefix of t' items of I'_ℓ and $I'_\ell(t'')$ as the prefix of t'' items of I'_ℓ . We have that

$$\begin{aligned}
 \text{Opt}(I_\ell(1, t)) &\leq \text{Opt}(I'_\ell(t')) && \text{(from Equation (3.2))} \\
 &\leq \text{Opt}(I'_\ell(t'')) + (t' - t'') \\
 &\leq \text{Opt}(I'_\ell(t'')) + 2\mathbb{E}[\mathcal{W}(I)]^{2/3}. && \text{(from Equation (3.4))}
 \end{aligned}$$

Further, from Equation (3.3) we also have

$$\text{Opt}(I'_\ell) \leq \text{Opt}(I_\ell).$$

Therefore,

$$\begin{aligned}
 \text{Opt}(I_\ell(1, t)) &\leq \text{Opt}(I'_\ell(t'')) + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \frac{t''}{n'} \mathbb{E} [\text{Opt}(I'_\ell)] + C_1 \sqrt{n'} (\log n')^{3/4} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \quad (\text{using Lemma 3.3}) \\
 &= \frac{t}{n} \mathbb{E} [\text{Opt}(I'_\ell)] + C_1 \sqrt{n'} (\log n')^{3/4} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \frac{t}{n} \mathbb{E} [\text{Opt}(I_\ell)] + C_1 (n')^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \frac{t}{n} \mathbb{E} [\text{Opt}(I_\ell)] + C_1 \mathbb{E} [|I_\ell|]^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3}.
 \end{aligned}$$

We now use the facts that $\text{Opt}(I_\ell) \leq \text{Opt}(I)$ and $\delta |I_\ell| \leq \mathcal{W}(I)$ to obtain

$$\text{Opt}(I(1, t)) = \text{Opt}(I_\ell(1, t)) \leq \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + C'_1 \mathbb{E} [\text{Opt}(I)]^{2/3} \quad (3.5)$$

with probability at least $1 - \exp(-a''(\log \mathbb{E} [\text{Opt}(I)])^{1/3})$ for some constants $a'', C'_1 > 0$. This ends Case 2.1.

Case 2.2. $t' > n'$.

This is the easier case. Since we have $t' > n'$, by the definitions of t', n' , we obtain that

$$\frac{t}{n} \mathbb{E} [|I_\ell|] + \mathbb{E} [\mathcal{W}(I)]^{2/3} > \mathbb{E} [|I_\ell|] - \mathbb{E} [\mathcal{W}(I)]^{2/3}.$$

This gives us the inequality

$$\left(1 - \frac{t}{n}\right) \mathbb{E} [|I_\ell|] < 2\mathbb{E} [\mathcal{W}(I)]^{2/3}. \quad (3.6)$$

Further observe that

$$\begin{aligned}
 |I_\ell(1, t)| &\leq t' && (\text{by Equation (3.2)}) \\
 &\leq n' + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} && (\text{by definition of } n') \\
 &= |I'_\ell| + 2\mathbb{E} [\mathcal{W}(I)]^{2/3}. && (\text{from Equation (3.3)})
 \end{aligned}$$

This implies that

$$\begin{aligned}
 \text{Opt}(I_\ell(1, t)) &\leq \text{Opt}(I'_\ell) + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \mathbb{E} [\text{Opt}(I'_\ell)] + C_1 \sqrt{n'} (\log n')^{3/4} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \quad (\text{using Lemma 3.3}) \\
 &\leq \mathbb{E} [\text{Opt}(I'_\ell)] + C_1 (n')^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \mathbb{E} [\text{Opt}(I'_\ell)] + C_1 (\mathbb{E} [|I_\ell|])^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \quad (\text{since } n' \leq \mathbb{E} [|I_\ell|]) \\
 &\leq \mathbb{E} [\text{Opt}(I_\ell)] + C_1 (\mathbb{E} [|I_\ell|])^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &= \frac{t}{n} \mathbb{E} [\text{Opt}(I_\ell)] + \left(1 - \frac{t}{n}\right) \mathbb{E} [\text{Opt}(I_\ell)] + C_1 (\mathbb{E} [|I_\ell|])^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \frac{t}{n} \mathbb{E} [\text{Opt}(I_\ell)] + \left(1 - \frac{t}{n}\right) \mathbb{E} [|I_\ell|] + C_1 (\mathbb{E} [|I_\ell|])^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} \\
 &\leq \frac{t}{n} \mathbb{E} [\text{Opt}(I_\ell)] + 2\mathbb{E} [\mathcal{W}(I)]^{2/3} + C_1 \mathbb{E} [|I_\ell|]^{2/3} + 2\mathbb{E} [\mathcal{W}(I)]^{2/3}. \\
 &\hspace{15em} (\text{using Equation (3.6)})
 \end{aligned}$$

We again use the facts that $\text{Opt}(I_\ell) \leq \text{Opt}(I)$ and $\delta |I_\ell| \leq \mathcal{W}(I)$ to obtain

$$\text{Opt}(I(1, t)) = \text{Opt}(I_\ell(1, t)) \leq \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + C'_1 \mathbb{E} [\text{Opt}(I)]^{2/3} \quad (3.7)$$

with probability at least $1 - \exp(-a''(\log \mathbb{E} [\text{Opt}(I)])^{1/3})$ for some constants $a'', C'_1 > 0$. This ends Case 2.2, and hence the analysis in the scenario when the event E happens with probability at least $\exp(-\frac{1}{2}a'(\log \mathbb{E} [\text{Opt}(I)])^{3/2})$.

On the other hand, suppose the event E happens with probability at most $\exp(-\frac{1}{2}a'(\log \mathbb{E} [\text{Opt}(I)])^{3/2})$. In other words, with probability at least $1 - \exp(-\frac{1}{2}a'(\log \mathbb{E} [\text{Opt}(I)])^{3/2})$, we open new bins while packing $I_s(1, t)$. Then after the final packing, every bin (except possibly one) is filled to a level of at least $1 - \delta$. Hence, in this case,

$$\begin{aligned} \text{Opt}(I(1, t)) &\leq \frac{1}{1 - \delta} \mathcal{W}(I(1, t)) + 1 \\ &\leq (1 + 2\delta) \frac{t}{n} \mathbb{E} [\mathcal{W}(I)] + (1 + 2\delta) \mathbb{E} [\mathcal{W}(I)]^{2/3} + 1 \quad (\text{using Lemma 3.4}) \\ &\leq (1 + 2\delta) \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} \end{aligned} \quad (3.8)$$

with probability at least $1 - \exp(-a''' \mathbb{E} [\text{Opt}(I)]^{1/3})$ for some constants $a''', C_2 > 0$.

Therefore, if the event E occurs with probability at least $\exp(-\frac{1}{2}a' \mathbb{E} [\text{Opt}(I)]^{1/3})$, then Equation (3.7) applies. Otherwise, Equation (3.8) applies. Hence, we obtain that there exist some constants a, C such that

$$\text{Opt}(I(1, t)) \leq (1 + 2\delta) \frac{t}{n} \mathbb{E} [\text{Opt}(I)] + C \mathbb{E} [\text{Opt}(I)]^{2/3}$$

holds with probability at least $1 - \exp(-a \mathbb{E} [\text{Opt}(I)]^{1/3})$.

This completes the proof. $\square$

3.1.2 Analysis of Blueprint Packing. Using the above lemmas, we will now analyze Algorithm 2 to bound the number of bins used to pack J_2 . Let us denote the number of bins used by Algorithm 2 with $\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1)$. We interpret this notation as the number of bins used to pack J_2 using the blueprint packing $\mathcal{A}_\alpha(J_1)$. Let $\text{BlueP}_{\text{unmatch}}$ denote the number of bins opened for the large items in J_2 for which we could not find a proxy item to be replaced. (See line 16 in Algorithm 2.) Let $\text{BlueP}_{\text{small}}$ denote the number of bins opened because a small item could not be packed in the S -slots using NF. (See line 21 in Algorithm 2.) Even before the arrival of the set J_2 , we have used $\mathcal{A}_\alpha(J_1)$ number of bins to construct the blueprint packing. Hence,

$$\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1) = \mathcal{A}_\alpha(J_1) + \text{BlueP}_{\text{unmatch}} + \text{BlueP}_{\text{small}}. \quad (3.9)$$

We will bound each of the summands in the right hand side of the above equation. First, since $\mathcal{A}_\alpha$ is a BP algorithm with AAR α , we have

$$\mathcal{A}_\alpha(J_1) \leq \alpha \text{Opt}(J_1) + o(\text{Opt}(J_1)). \quad (3.10)$$

To bound $\text{BlueP}_{\text{unmatch}}$, we use the upright matching result of Lemma 2.1. Let $L(J_1)$ and $L(J_2)$ denote the set of large items in the sequences J_1 and J_2 , respectively, and let $\kappa_1 := |L(J_1)|$ and $\kappa_2 := |L(J_2)|$. Using Lemma 3.5, we get that, w.h.p., $\kappa_1 \geq \mathbb{E} [\kappa_1] - \mathbb{E} [\mathcal{W}(J_1)]^{2/3}$, and $\kappa_2 \leq \mathbb{E} [\kappa_2] + \mathbb{E} [\mathcal{W}(J_2)]^{2/3}$. Since $|J_1| = |J_2|$ and since each item in $J_1 \cup J_2$ is sampled independently and identically, we have that $\mathbb{E} [\kappa_1] = \mathbb{E} [\kappa_2]$ and $\mathbb{E} [\mathcal{W}(J_1)] = \mathbb{E} [\mathcal{W}(J_2)]$. Hence, with high probability,

$$\kappa_2 \leq \kappa_1 + 2\mathbb{E} [\mathcal{W}(J_1)]^{2/3}. \quad (3.11)$$

Let $L(J_1) = \{r_1, r_2, \dots, r_{\kappa_1}\}$ and $L(J_2) = \{s_1, s_2, \dots, s_{\kappa_2}\}$. According to lines 10–17 of Algorithm 2, each s_i ($i \in [\kappa_1]$) is packed in the place of an r_j ($j \in [\kappa_1]$) satisfying $r_j > s_i$ such that r_j is minimum. Now, if we consider the sets of points $P^- = \{(-1, r_j)\}_{j \in [\kappa_1]}$, and $P^+ = \{(+1, s_i)\}_{i \in [\kappa_1]}$, then one can see that this procedure of packing s_i -s in place of r_j -s is exactly the same as the maximum upright matching procedure detailed in Algorithm 1. Hence, by Lemma 2.1 and also accounting for the

items in $\{s_{\kappa_1+1}, s_{\kappa_1+2}, \dots, s_{\kappa_2}\}$, we obtain, w.h.p., an upper bound on $\text{BlueP}_{\text{unmatch}}$, the number of new bins opened while packing $L(J_2)$:

$$\text{BlueP}_{\text{unmatch}} \leq \kappa_2 - \kappa_1 + K\sqrt{\kappa_1}(\log \kappa_1)^{3/4}.$$

Using Equation (3.11), we finally obtain the following bound on $\text{BlueP}_{\text{unmatch}}$ that holds with high probability:

$$\text{BlueP}_{\text{unmatch}} \leq 2\mathbb{E}[\mathcal{W}(J_1)]^{2/3} + K\sqrt{\kappa_1}(\log \kappa_1)^{3/4}.$$

We will simplify the above inequality further. Each large item has size at least δ . Hence, at most $1/\delta$ number of large items can be fit in a bin. Therefore, $\text{Opt}(J_1) \geq \delta\kappa_1$. Also, $\text{Opt}(J_1) \geq \mathcal{W}(J_1)$. Since δ is a constant, and since $\log a \leq a^{2/9}$ for any positive integer a , we have that for some constant K_1 , w.h.p.,

$$\text{BlueP}_{\text{unmatch}} \leq 2\mathbb{E}[\text{Opt}(J_1)]^{2/3} + K_1\text{Opt}(J_1)^{2/3}.$$

Using Lemma 3.6, we see that, w.h.p., $\text{Opt}(J_1) \leq (1 + 2\delta)\mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)])$. Hence, the above inequality can be transformed as

$$\text{BlueP}_{\text{unmatch}} = o(\mathbb{E}[\text{Opt}(J_1)]). \quad (3.12)$$

The only part that is left to be bounded is $\text{BlueP}_{\text{small}}$, the number of bins opened because the small items in J_2 could not be packed in the S -slots (lines 20–24 of Algorithm 2). We will bound this by using the concentration of weights of small items in J_1 and J_2 . For this purpose, let us write $S(J_1), S(J_2)$ to denote the sets of small items in the sets J_1, J_2 , respectively. Also, let $\mathcal{S}_e$ denote the set of existing S -slots before we start packing the items in J_2 , i.e., the set of S -slots created just after the for loop in lines 3–7 of Algorithm 2 ends. Similarly, let $\mathcal{S}_n$ denote the set of S -slots created during the process of packing J_2 , i.e., when line 21 of Algorithm 2 is executed. We would like to bound $|\mathcal{S}_n|$. To this end, let us define the volume of a given set $\mathcal{S}$ of S -slots as the sum of the size³ of each S -slot in $\mathcal{S}$. Let $\text{vol}(\mathcal{S})$ denote the volume of a given set $\mathcal{S}$ of S -slots. Since we packed the small items in J_2 in the set of S -slots given by $\mathcal{S}_e \cup \mathcal{S}_n$ using NF, each of these S -slots, except possibly one, will have an unused volume of at most δ . Hence,

$$\begin{aligned} \mathcal{W}(S(J_2)) &\geq \text{vol}(\mathcal{S}_n \cup \mathcal{S}_e) - \delta|\mathcal{S}_n \cup \mathcal{S}_e| - 1 \\ &= \text{vol}(\mathcal{S}_n) + \text{vol}(\mathcal{S}_e) - \delta|\mathcal{S}_n| - \delta|\mathcal{S}_e| - 1 \\ &= |\mathcal{S}_n| + \text{vol}(\mathcal{S}_e) - \delta|\mathcal{S}_n| - \delta|\mathcal{S}_e| - 1 \quad (\text{since each } S\text{-slot in } \mathcal{S}_n \text{ has unit volume}) \\ &= \text{vol}(\mathcal{S}_e) + (1 - \delta)|\mathcal{S}_n| - \delta|\mathcal{S}_e| - 1. \end{aligned} \quad (3.13)$$

Let us look at the right hand side of the last inequality closely. The number of S -slots in $\mathcal{S}_e$ is upper bounded by the number of bins used by the algorithm $\mathcal{A}_\alpha$ to pack J_1 . At the same time, $\text{vol}(\mathcal{S}_e)$ is lower bounded by the volume of the small items in J_1 . Hence, Equation (3.13) can be rewritten as

$$\begin{aligned} \mathcal{W}(S(J_2)) &\geq \text{vol}(\mathcal{S}_e) + (1 - \delta)|\mathcal{S}_n| - \delta|\mathcal{S}_e| - 1 \\ &\geq \mathcal{W}(S(J_1)) + (1 - \delta)|\mathcal{S}_n| - \delta\mathcal{A}_\alpha(J_1) - 1 \\ &\geq \mathcal{W}(S(J_1)) + (1 - \delta)|\mathcal{S}_n| - \delta\alpha\text{Opt}(J_1) - o(\text{Opt}(J_1)). \end{aligned}$$

Rearranging terms on both the sides, we obtain an upper bound on $|\mathcal{S}_n|$:

$$\text{BlueP}_{\text{small}} = |\mathcal{S}_n| \leq \frac{1}{1 - \delta} \left(\mathcal{W}(S(J_2)) - \mathcal{W}(S(J_1)) \right) + \delta\alpha\text{Opt}(J_1) + o(\text{Opt}(J_1)). \quad (3.14)$$

³The size of an S -slot is the maximum size of an item that can fit in that slot.

The only task left is to bound the quantity $\mathcal{W}(S(J_2)) - \mathcal{W}(S(J_1))$. Intuitively, this quantity should not be too big since the joint distributions of J_2 and J_1 are exactly the same. This is formalized in the next claim.

CLAIM 3. *With probability at least $1 - 4 \exp\left(-\frac{3}{8} \mathbb{E} [\text{Opt}(J_1)]^{1/3}\right)$, we have*

$$\mathcal{W}(S(J_2)) - \mathcal{W}(S(J_1)) \leq 2\mathbb{E} [\text{Opt}(J_1)]^{2/3}.$$

PROOF. For any item x , define the random variable S_x which takes value 0 if x is large, and takes value x if x is small. Now, observe that $\sum_{x \in J_1} S_x = \mathcal{W}(S(J_1))$ and $\sum_{x \in J_2} S_x = \mathcal{W}(S(J_2))$. Moreover, since J_1 and J_2 each contain k items and since each item is sampled independently from the same distribution, it must be the case that $\mathbb{E} [\mathcal{W}(S(J_1))] = \mathbb{E} [\mathcal{W}(S(J_2))]$. The same argument implies that $\mathbb{E} [\text{Opt}(J_1)] = \mathbb{E} [\text{Opt}(J_2)]$.

Applying Bernstein's inequality (Lemma 3.2) on the set of random variables $\{S_x\}_{x \in J_2}$, we obtain that

$$\begin{aligned} \mathbb{P} \left[\mathcal{W}(S(J_2)) - \mathbb{E} [\mathcal{W}(S(J_2))] \geq \mathbb{E} [\text{Opt}(J_2)]^{2/3} \right] &= \mathbb{P} \left[\sum_{x \in J_2} S_x - \sum_{x \in J_2} \mathbb{E} [S_x] \geq \mathbb{E} [\text{Opt}(J_2)]^{2/3} \right] \\ &\leq 2 \exp \left(- \frac{\mathbb{E} [\text{Opt}(J_2)]^{4/3}}{2 \left(\mathbb{E} [\mathcal{W}(S(J_2))] + \frac{1}{3} \mathbb{E} [\text{Opt}(J_2)]^{2/3} \right)} \right). \end{aligned}$$

Now, observe that $\mathcal{W}(S(J_2)) \leq \mathcal{W}(J_2) \leq \text{Opt}(J_2)$. Also, $\mathbb{E} [\text{Opt}(J_2)]^{2/3} \leq \mathbb{E} [\text{Opt}(J_2)]$. Hence, the above inequality can be simplified as

$$\mathbb{P} \left[\mathcal{W}(S(J_2)) - \mathbb{E} [\mathcal{W}(S(J_2))] \geq \mathbb{E} [\text{Opt}(J_2)]^{2/3} \right] \leq 2 \exp \left(- \frac{3}{8} \mathbb{E} [\text{Opt}(J_2)]^{1/3} \right).$$

In the same manner, we can apply Bernstein's inequality on the set of random variables $\{S_x\}_{x \in J_1}$ to obtain that

$$\mathbb{P} \left[\mathcal{W}(S(J_1)) - \mathbb{E} [\mathcal{W}(S(J_1))] \leq -\mathbb{E} [\text{Opt}(J_1)]^{2/3} \right] \leq 2 \exp \left(- \frac{3}{8} \mathbb{E} [\text{Opt}(J_1)]^{1/3} \right).$$

Therefore, with probability at least $1 - 4 \exp \left(- \frac{3}{8} \mathbb{E} [\text{Opt}(J_1)]^{1/3} \right)$, both the following inequalities hold:

$$\begin{aligned} \mathcal{W}(S(J_2)) - \mathbb{E} [\mathcal{W}(S(J_2))] &\leq \mathbb{E} [\text{Opt}(J_2)]^{2/3}, \\ \mathcal{W}(S(J_1)) - \mathbb{E} [\mathcal{W}(S(J_1))] &\geq -\mathbb{E} [\text{Opt}(J_1)]^{2/3}. \end{aligned}$$

Since $\mathbb{E} [\mathcal{W}(S(J_1))] = \mathbb{E} [\mathcal{W}(S(J_2))]$ and $\mathbb{E} [\text{Opt}(J_1)] = \mathbb{E} [\text{Opt}(J_2)]$, we obtain that with probability at least $1 - 4 \exp \left(- \frac{3}{8} \mathbb{E} [\text{Opt}(J_1)]^{1/3} \right)$,

$$\mathcal{W}(S(J_2)) - \mathcal{W}(S(J_1)) \leq 2\mathbb{E} [\text{Opt}(J_1)]^{2/3}.$$

This ends the proof of the claim. $\square$

Using the above claim with Equation (3.14), we can obtain an upper bound on $\text{BlueP}_{\text{small}}$ that holds with high probability as follows:

$$\begin{aligned}\text{BlueP}_{\text{small}} &\leq \frac{1}{1-\delta} \left(2\mathbb{E} [\text{Opt}(J_1)]^{2/3} \right) + \delta\alpha\text{Opt}(J_1) + o(\text{Opt}(J_1)) \\ &\leq o(\mathbb{E} [\text{Opt}(J_1)]) + \delta\alpha\text{Opt}(J_1) + o(\text{Opt}(J_1)).\end{aligned}$$

Lemma 3.6 with $t = n$ tells us that $\text{Opt}(J_1) \leq (1 + 2\delta)\mathbb{E} [\text{Opt}(J_1)] + o(\mathbb{E} [\text{Opt}(J_1)])$. Hence, we transform the above inequality to obtain the following upper bound on $\text{BlueP}_{\text{small}}$:

$$\text{BlueP}_{\text{small}} \leq \delta\alpha\text{Opt}(J_1) + o(\mathbb{E} [\text{Opt}(J_1)]). \quad (3.15)$$

Summing up Equations (3.10), (3.12), and (3.15), we obtain an upper bound on $\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1)$ (Equation (3.9)), the number of bins used to pack J_2 using Algorithm 2:

$$\begin{aligned}\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1) &= \mathcal{A}_\alpha(J_1) + \text{BlueP}_{\text{unmatch}} + \text{BlueP}_{\text{small}} \\ &\leq \alpha\text{Opt}(J_1) + \alpha\delta\text{Opt}(J_1) + o(\mathbb{E} [\text{Opt}(J_1)]) \\ &= \alpha(1 + \delta)\text{Opt}(J_1) + o(\mathbb{E} [\text{Opt}(J_1)]).\end{aligned}$$

Using Lemma 3.6 with $t = n$, we can see that $\text{Opt}(J_1) \leq (1 + 2\delta)\mathbb{E} [\text{Opt}(J_1)] + o(\mathbb{E} [\text{Opt}(J_1)])$. Moreover, we know that $\mathbb{E} [\text{Opt}(J_1)] = \mathbb{E} [\text{Opt}(J_2)]$. Therefore, we can transform the above inequality to obtain the following form for the upper bound on $\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1)$:

$$\begin{aligned}\text{BlueP}(J_2, \mathcal{A}_\alpha, J_1) &\leq \alpha(1 + \delta)(1 + 2\delta)\mathbb{E} [\text{Opt}(J_1)] + o(\mathbb{E} [\text{Opt}(J_1)]) \\ &\leq \alpha(1 + 4\delta)\mathbb{E} [\text{Opt}(J_1)] + o(\mathbb{E} [\text{Opt}(J_1)]) \\ &= \alpha(1 + 4\delta)\mathbb{E} [\text{Opt}(J_2)] + o(\mathbb{E} [\text{Opt}(J_2)]).\end{aligned} \quad (3.16)$$

In other words, using an α -approximate offline algorithm to pack J_1 , we can pack J_2 in an *online* manner while achieving (essentially) the same approximation factor.

3.2 Algorithm Assuming That the Value of n Is Known

We now describe our algorithm which assumes the knowledge of the number of items. The input I is denoted by the list $x_1, x_2, \dots, x_n$. We partition the entire input into $m := 1/\delta^2$ stages $T_0, T_1, \dots, T_{m-1}$. The zeroth stage T_0 , called the *sampling stage*, contains the first $\delta^2 n$ items, i.e., $x_1, x_2, \dots, x_{\delta^2 n}$. For $j \in [m-1]$, the stage T_j contains the items with index starting from $j\delta^2 n + 1$ till $\min(n, (j+1)\delta^2 n)$. In essence, T_0 contains the first $\delta^2 n$ items, T_1 contains the next $\delta^2 n$ items, T_2 contains the next $\delta^2 n$ items, and so on. Note that the number of stages m is a constant, and the last stage may contain fewer than $\delta^2 n$ items. In any stage T_j , we denote the set of large items and small items by L_j and S_j , respectively. Note that for any $j \in [m-2]$, $|T_j| = |T_{j+1}|$ and since all the items are sampled independently from the same distribution, we know that the joint distributions of the sets T_j and T_{j+1} are the same. Hence, to pack T_j , we can first pack T_{j+1} using a good offline approximation algorithm $\mathcal{A}_\alpha$ and use that packing as a blueprint. The last stage T_{m-1} may contain fewer than $\delta^2 n$ items. However, its size is only a small fraction in comparison to the entire input; hence, we can show that it does not affect the final packing much.

The algorithm is as follows. First we pack T_0 , the sampling stage, using NF. The sampling stage contains only a small but a constant fraction of the entire input set; hence it uses only a few number of bins when compared to the final packing but at the same time provides a good estimate of the underlying distribution. If $|L_0|$, the number of large items in the sampling stage, is at most $\delta^3 \mathcal{W}(T_0)$, then we continue using NF for the rest of the entire input too. Intuitively, NF performs well in this case as most of the items are small. Thus, from now on, let us assume that $|L_0| > \delta^3 \mathcal{W}(T_0)$. Consider an intermediate point when all the items in the stage T_{j-1} have arrived and the first element of stage T_j is about to arrive ($j \geq 1$). At this point, we compute the packing $\mathcal{A}_\alpha(T_{j-1})$ and use it as a

Algorithm 3: Alg($x_1, x_2, \dots, x_n$): An Algorithm for Online BP under the i.i.d. Model Assuming That the Number of Items n Is Known Before-Hand

Input: $I = \{x_1, x_2, \dots, x_n\}$ where each x_i ($i \in [n]$) is sampled independently and identically.

```

1: Initialize  $m := \frac{1}{\delta^2}$  ▷ Number of stages
2: for  $j$  in  $\{0, 1, \dots, m-1\}$  do
3:   Define  $T_j$  to be the sequence  $x_{j\delta^2 n+1}, x_{j\delta^2 n+2}, \dots, x_{\min\{n, (j+1)\delta^2 n\}}$  ▷ The  $j^{\text{th}}$  stage
4: end for
5: Pack the sampling stage  $T_0$  using Next-Fit.
6: if  $|L_0| \leq \delta^3 \mathcal{W}(T_0)$  then ▷ Very few large items in the sampling stage
7:   Use Next-Fit for all the remaining stages  $T_1, T_2, \dots, T_{m-1}$ .
8: else
9:   for  $j = 1$  to  $m-1$  do
10:    Pack the stage  $T_j$  using the blueprint packing procedure BlueP( $T_j, \mathcal{A}_\alpha, T_{j-1}$ ).
11:   end for
12: end if

```

blueprint to pack T_j , i.e., we run BlueP($T_j, \mathcal{A}_\alpha, T_{j-1}$) (Algorithm 2) to pack T_j . In this way, we pack all the stages. We call this algorithm Alg. Algorithm 3 gives a more formal pseudocode of Alg.

We will proceed to analyze the algorithm Alg. We split the analysis into the following two cases depending on the truth value of the if condition on line 6 of Algorithm 3: when $|L_0| \leq \delta^3 \cdot \mathcal{W}(T_0)$ and when $|L_0| > \delta^3 \cdot \mathcal{W}(T_0)$.

3.2.1 Case 1: $|L_0| \leq \delta^3 \cdot \mathcal{W}(T_0)$. Recall that in this case, we just continue with NF for all the remaining items. To bound the NF solution, we first consider the number of bins that contain at least one large item. For this, we bound the value of $|I_\ell|$. Then we consider the bins that contain only small items and bound this value in terms of weight of all items $\mathcal{W}(I)$.

CLAIM 4. *With probability at least $1 - 4 \exp\left(-\frac{\delta}{6} \mathbb{E}[\text{Opt}(I)]^{1/3}\right)$, for some positive constant a , we have that*

$$|I_\ell| \leq \delta \cdot \mathcal{W}(T_0) + a \mathbb{E}[\text{Opt}(I)]^{2/3}.$$

PROOF. As the sampling stage contains $\delta^2 n$ items, $\mathbb{E}[|L_0|] = \delta^2 \mathbb{E}[|I_\ell|]$. Two applications of Lemma 3.5 give us the following inequalities, we have

$$\begin{aligned} \mathbb{P}\left[|L_0| \leq \delta^2 \mathbb{E}[|I_\ell|] - \mathbb{E}[\mathcal{W}(I)]^{2/3}\right] &\leq 2 \exp\left(-\frac{\delta}{3} \mathbb{E}[\mathcal{W}(I)]^{1/3}\right) \text{ and} \\ \mathbb{P}\left[|I_\ell| \geq \mathbb{E}[|I_\ell|] + \mathbb{E}[\mathcal{W}(I)]^{2/3}\right] &\leq 2 \exp\left(-\frac{\delta}{3} \mathbb{E}[\mathcal{W}(I)]^{1/3}\right). \end{aligned}$$

From the above inequalities we have that

$$|I_\ell| \leq \frac{1}{\delta^2} |L_0| + \left(1 + \frac{1}{\delta^2}\right) \mathbb{E}[\mathcal{W}(I)]^{2/3}$$

with probability at least $1 - 4 \exp\left(-\frac{\delta}{3} \mathbb{E}[\mathcal{W}(I)]^{1/3}\right)$. We can use the inequalities $2\mathcal{W}(I) \geq \text{Opt}(I) \geq \mathcal{W}(I)$ and $|L_0| \leq \delta^3 \cdot \mathcal{W}(T_0)$ to conclude the proof of this claim. $\square$

The number of bins that contain at least one large item is upper bounded by $|I_\ell|$. Now we bound the number of bins that contain only small items. Note that NF fills each such bin (with at most one possible exception) up to a capacity at least $(1 - \delta)$. So, the number of bins containing only

small items is at most $\frac{1}{1-\delta}\mathcal{W}(I) + 1$. Thus, with high probability,

$$\begin{aligned} \text{NF}(I) &\leq |I_\ell| + \frac{1}{(1-\delta)}\mathcal{W}(I) + 1 \\ &\leq |I_\ell| + (1+2\delta)\mathcal{W}(I) + 1 \\ &\leq \delta \cdot \mathcal{W}(T_0) + (1+2\delta)\mathcal{W}(I) + a\mathbb{E}[\text{Opt}(I)]^{2/3} + 1 \end{aligned}$$

for some constant a .

Using Lemma 3.4, we get that, with high probability, $\mathcal{W}(I) \leq \mathbb{E}[\mathcal{W}(I)] + \mathbb{E}[\mathcal{W}(I)]^{2/3}$. Using the facts $\mathcal{W}(T_0) \leq \mathcal{W}(I)$ and $\text{Opt}(I)/2 \leq \mathcal{W}(I) \leq \text{Opt}(I)$, we get

$$\text{Alg}(I) = \text{NF}(I) \leq (1+3\delta)\mathbb{E}[\text{Opt}(I)] + a_1\mathbb{E}[\text{Opt}(I)]^{2/3}, \quad (3.17)$$

with high probability for some constant $a_1 > 0$.

3.2.2 Case 2: $|L_0| > \delta^3 \cdot \mathcal{W}(T_0)$. We split our analysis in this case into two parts. We first analyze the number of bins used in the sampling stage T_0 and then analyze the number of bins used in the remaining stages.

Using Lemma 3.5, we obtain w.h.p. that $|L_0| \leq \delta^2\mathbb{E}[|I_\ell|] + \mathbb{E}[\mathcal{W}(I)]^{2/3}$. Hence,

$$\begin{aligned} \mathbb{E}[|I_\ell|] &\geq \frac{1}{\delta^2}|L_0| - \frac{1}{\delta^2}\mathbb{E}[\mathcal{W}(I)]^{2/3} \\ &\geq \delta\mathcal{W}(T_0) - \frac{1}{\delta^2}\mathbb{E}[\mathcal{W}(I)]^{2/3}. \end{aligned} \quad (3.18)$$

From Lemma 3.4, since $|T_0| = \delta^2|I|$, we obtain that the inequality $\mathcal{W}(T_0) \geq \delta^2\mathbb{E}[\mathcal{W}(I)] - \mathbb{E}[\mathcal{W}(I)]^{2/3}$ holds with high probability. Substituting this inequality in Equation (3.18), we obtain that

$$\mathbb{E}[|I_\ell|] \geq \delta^3\mathbb{E}[\mathcal{W}(I)] - \left(\delta + \frac{1}{\delta^2}\right)\mathbb{E}[\mathcal{W}(I)]^{2/3}. \quad (3.19)$$

For any $j \geq 1$, using the fact that $|T_j| = \delta^2|I|$ and using Lemma 3.5, we obtain the inequality $|L_j| \geq \delta^2\mathbb{E}[|I_\ell|] - \mathbb{E}[\mathcal{W}(I)]^{2/3}$ that holds with high probability. Substituting Equation (3.19) in this inequality, we obtain, w.h.p.,

$$|L_j| \geq \delta^5\mathbb{E}[\mathcal{W}(I)] - (2 + \delta^3)\mathbb{E}[\mathcal{W}(I)]^{2/3}. \quad (3.20)$$

Each of the Equations (3.19) and (3.20) holds with high probability.

Note that $\mathcal{W}(I) \geq \text{Opt}(I)/2$. So from now on, we assume that there exist constants $C_1, C_2 > 0$ which depend on δ such that, w.h.p., both the following inequalities hold:

$$\mathbb{E}[|I_\ell|] \geq C_1 \cdot \mathbb{E}[\text{Opt}(I)]. \quad (3.21)$$

$$|L_j| \geq C_2 \cdot \mathbb{E}[\text{Opt}(I)]. \quad (3.22)$$

— *Analysis of the Sampling Stage:* Recall that the number of items considered in the sampling stage is δ^2n . We will bound the number of large items and the weight of items in this stage using Bernstein's inequality.

- (1) Since sampling stage has δ^2n items, $\mathbb{E}[|L_0|] = \delta^2\mathbb{E}[|I_\ell|]$. By applying Bernstein's inequality for $X_1, X_2, \dots, X_{|T_0|}$ where X_i takes value 1 if x_i is large and 0 otherwise,

we get

$$\begin{aligned}
 \mathbb{P}[|L_0| \geq 2\delta^2 \mathbb{E}[|I_\ell|]] &= \mathbb{P}[|L_0| \geq \mathbb{E}[|L_0|] + \delta^2 \mathbb{E}[|I_\ell|]] \\
 &\leq 2 \exp\left(-\frac{\delta^4 \mathbb{E}[|I_\ell|]^2}{2\mathbb{E}[|L_0|] + \frac{2}{3}\delta^2 \mathbb{E}[|I_\ell|]}\right) \\
 &\leq 2 \exp\left(-\frac{1}{3}\delta^2 \mathbb{E}[|I_\ell|]\right) \\
 &\leq 2 \exp(-a_2 \cdot \mathbb{E}[\text{Opt}(I)]) \quad (\text{from Equation (3.21)})
 \end{aligned}$$

for some constant $a_2 > 0$. So, with high probability,

$$\begin{aligned}
 |L_0| &\leq 2\delta^2 \mathbb{E}[|I_\ell|] \\
 &\leq 2\delta \mathbb{E}[\text{Opt}(I)].
 \end{aligned} \tag{3.23}$$

- (2) Similarly, $\mathbb{E}[\mathcal{W}(T_0)] = \delta^2 \mathbb{E}[\mathcal{W}(I)]$. By applying Bernstein's inequality for $X_1, X_2, \dots, X_{|T_0|}$ where X_i takes value x_i , we get

$$\begin{aligned}
 \mathbb{P}[\mathcal{W}(T_0) \geq 2\delta^2 \mathbb{E}[\mathcal{W}(I)]] &= \mathbb{P}[\mathcal{W}(T_0) \geq \mathbb{E}[\mathcal{W}(T_0)] + \delta^2 \mathbb{E}[\mathcal{W}(I)]] \\
 &\leq 2 \exp\left(\frac{-\delta^4 \mathbb{E}[\mathcal{W}(I)]^2}{2\delta^2 \mathbb{E}[\mathcal{W}(T_0)] + \frac{2}{3}\delta^2 \mathbb{E}[\mathcal{W}(I)]}\right) \\
 &\leq 2 \exp\left(\frac{-\delta^2}{3} \mathbb{E}[\mathcal{W}(I)]\right) \leq 2 \exp\left(\frac{-\delta^2 \mathbb{E}[\text{Opt}(I)]}{6}\right).
 \end{aligned}$$

So, with high probability we have

$$\begin{aligned}
 \mathcal{W}(T_0) &\leq 2\delta^2 \mathbb{E}[\mathcal{W}(I)] \\
 &\leq 2\delta^2 \mathbb{E}[\text{Opt}(I)].
 \end{aligned} \tag{3.24}$$

Among the $\text{NF}(T_0)$ number of bins used by NF to pack the sampling stage, the number of bins that contain at least one large item is at most $|L_0|$. On the other hand, each bin (with at most one exception) that contains only small items is filled up to a volume of at least $(1 - \delta)$. Hence,

$$\begin{aligned}
 \text{NF}(T_0) &\leq |L_0| + \frac{1}{1 - \delta} \mathcal{W}(S_0) + 1 \\
 &\leq |L_0| + \frac{1}{1 - \delta} \mathcal{W}(T_0) + 1.
 \end{aligned}$$

Substituting Equations (3.23) and (3.24) in the above inequality, we obtain that the following inequality holds with high probability:

$$\begin{aligned}
 \text{NF}(T_0) &\leq 2\delta \mathbb{E}[\text{Opt}(I)] + \frac{2\delta^2}{1 - \delta} \mathbb{E}[\text{Opt}(I)] \\
 &\leq 4\delta \mathbb{E}[\text{Opt}(I)].
 \end{aligned} \tag{3.25}$$

— *Analysis of the Remaining Stages:* Recall that we pack each of the remaining stages using the blueprint packing $\mathcal{A}_\alpha(T_0)$. The analysis of the last stage T_{m-1} is slightly different as it is possible that $|T_{m-1}| < \delta^2 n = |T_0|$. Hence we analyze stages $T_1, T_2, \dots, T_{m-2}$ for now. At the end, we analyze the stage T_{m-1} . Consider any $j \in [m - 2]$. Lemma 3.1 tells us that $\text{BlueP}(T_j, \mathcal{A}_\alpha, T_{j-1})$, the number of bins required to pack T_j , is bounded, w.h.p., as follows:

$$\text{BlueP}(T_j, \mathcal{A}_\alpha, T_{j-1}) \leq \alpha(1 + 4\delta) \mathbb{E}[\text{Opt}(T_j)] + o(\mathbb{E}[\text{Opt}(T_j)]).$$

Now, we use Lemma 3.6 to obtain the following inequality that holds with high probability:

$$\begin{aligned}
 \text{BlueP}(T_j, \mathcal{A}_\alpha, T_{j-1}) &\leq \alpha(1+4\delta)(1+2\delta) \frac{|T_j|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(T_j)]) \\
 &\leq \alpha(1+4\delta)(1+2\delta) \frac{|T_j|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &\leq \alpha(1+15\delta) \frac{|T_j|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]).
 \end{aligned}$$

Since the above inequality holds with high probability and since there are only constant number of stages m , by union bound, the inequality obtained by summing the above inequality over all $j \in [m-2]$ also holds with high probability. In other words, with high probability, we obtain that

$$\begin{aligned}
 \sum_{j \in [m-2]} \text{BlueP}(T_j, \mathcal{A}_\alpha, T_{j-1}) &\leq \alpha(1+15\delta) \frac{\sum_{j=1}^{m-2} |T_j|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &\leq \alpha(1+15\delta) \frac{\sum_{j=0}^{m-1} |T_j|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &= \alpha(1+15\delta) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \tag{3.26}
 \end{aligned}$$

Now, we analyze the last stage T_{m-1} . Recall that Lemma 3.1 required the condition that $|J_1| = |J_2|$. Since T_{m-1} can have fewer items than stage T_{m-2} , we cannot use Lemma 3.1 directly. However, there is a simple workaround. Assume that, after the stage T_{m-1} ends, we sample $\delta n - |T_{m-1}|$ number of extra items. Let E denote this list of extra items. Let T'_{m-1} be the list of items T_{m-1} appended with the list E . Now, we can use Lemma 3.1 if we pack T'_{m-1} using the blueprint packing $\mathcal{A}_\alpha(T_{m-1})$ since $|T_{m-1}| = |T'_{m-1}|$. Since T_{m-1} is only a prefix of T'_{m-1} , we have that

$$\begin{aligned}
 \text{BlueP}(T_{m-1}, \mathcal{A}_\alpha, T_{m-2}) &\leq \text{BlueP}(T'_{m-1}, \mathcal{A}_\alpha, T_{m-2}) \\
 &\leq \alpha(1+4\delta) \mathbb{E} [\text{Opt}(T'_{m-1})] + o(\mathbb{E} [\text{Opt}(T'_{m-1})]) \\
 &= \alpha(1+4\delta) \mathbb{E} [\text{Opt}(T_{m-2})] + o(\mathbb{E} [\text{Opt}(T_{m-2})]) \\
 &\leq \alpha(1+4\delta)(1+2\delta) \frac{|T_{m-2}|}{n} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \quad (\text{using Lemma 3.6}) \\
 &= \alpha(1+4\delta)(1+2\delta) \delta^2 \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &\leq 2\alpha\delta^2 \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \tag{3.27}
 \end{aligned}$$

Adding up Equations (3.26) and (3.27) completes the analysis of the remaining stages.

$$\begin{aligned}
 \sum_{j \in [m-1]} \text{BlueP}(T_j, \mathcal{A}_\alpha, T_{j-1}) &\leq \alpha(1+15\delta+2\delta^2) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &\leq \alpha(1+16\delta) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \tag{3.28}
 \end{aligned}$$

We now combine the analyses of the sampling stage and the remaining stages. For the sampling stage, from Equation (3.25), we have $\text{NF}(T_0) \leq 4\delta \mathbb{E} [\text{Opt}(I)]$ with high probability. For all the remaining stages, Equation (3.28) gives the upper bound on the number of bins used. Combining both the results, we get an upper bound on $\text{Alg}(I)$ that holds w.h.p.

$$\begin{aligned}
 \text{Alg}(I) &\leq \alpha(1+20\delta) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\
 &\leq \alpha(1+\epsilon) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \tag{3.29}
 \end{aligned}$$

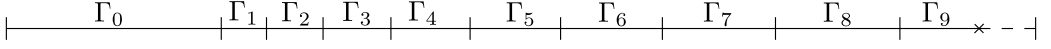


Fig. 1. The division of input into super-stages to get rid of the assumption on the knowledge of n . The $(j+1)$ th super-stage is denoted by Γ_j . Super-stage Γ_0 contains $n_0 = 1/\delta^3$ items, $\Gamma_0 \cup \Gamma_1$ contains $(1+\mu)n_0$ items, $\Gamma_0 \cup \Gamma_1 \cup \Gamma_2$ contains $(1+\mu)^2 n_0$ items, and so on. The last super-stage may not be full, but since it is very small in size compared to the entire input, it doesn't affect the performance of the algorithm.

In the low probability event when Equation (3.29) may not hold, we can bound $\text{Alg}(I)$ as follows. In the sampling stage, we have that $\text{NF}(T_0) \leq 2\text{Opt}(I) - 1$. For the remaining stages, we bound the number of bins containing at least one large item and the number of bins containing only small items. Each large item is considered at most twice in the packing: first—when it arrives in the input list, second—when it is in the sampling stage and thus takes the role of a proxy item. So, the number of bins containing at least one large item is at most $2|I_\ell|$. In each stage, with one possible exception, every bin opened which has only small items has an occupancy of at least $(1-\delta)$. Combining over all the stages, the number of bins which contain only small items is at most $\frac{1}{1-\delta}\mathcal{W}(I_s) + m$. Thus, we can bound the total number of bins used by Alg to be at most $2\text{Opt}(I) + 2|I_\ell| + \frac{1}{1-\delta}\mathcal{W}(I_s) + m$. On the other hand, we know that $\text{Opt}(I) \geq \mathcal{W}(I) \geq \delta|I_\ell| + \mathcal{W}(I_s)$. Hence, we obtain that $2|I_\ell| + \frac{1}{1-\delta}\mathcal{W}(I_s) \leq \frac{2}{\delta(1-\delta)}\text{Opt}(I)$. Combining all these, we obtain that

$$\text{Alg}(I) \leq \left(2 + \frac{2}{\delta(1-\delta)}\right) \text{Opt}(I) + m. \quad (3.30)$$

Now, to obtain the CR, suppose Equation (3.29) holds with probability $p (= 1 - o(1))$. We combine Equations (3.29) and (3.30) similar to the case when $|L_0| \leq \delta^3 \cdot \mathcal{W}(T_0)$:

$$\begin{aligned} \mathbb{E}[\text{Alg}(I)] &\leq p\left(\alpha(1+\varepsilon)\mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)])\right) \\ &\quad + (1-p)\left(\left(2 + \frac{2}{\delta(1-\delta)}\right)\mathbb{E}[\text{Opt}(I)] + m\right) \\ &\leq \alpha(1+\varepsilon)\mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)]). \quad (\text{since } 1-p = o(1) \text{ and } \delta \leq \varepsilon/16) \end{aligned}$$

Scaling the initial value ε to ε/α before the start of the algorithm, we obtain a CR of $\alpha + \varepsilon$.

3.3 Getting Rid of the Assumption on the Knowledge of the Input Size

In this subsection, we will extend Alg to devise an algorithm for online BP with i.i.d. items that guarantees essentially the same CR as Alg without knowing the value of n . We denote this algorithm by ImpAlg .

Let $\mu := \delta^2$. We first guess the value of n to be a constant $n_0 := 1/\delta^3$. Then, we run Alg until $\min\{n, n_0\}$ items arrive (here, if $\min\{n, n_0\} = n$, then it means that the input stream has ended before n_0 items have arrived). If $n > n_0$, i.e., if there are more items to arrive, then we revise our estimate of n by multiplying it with $(1+\mu)$, i.e., the new value of n is set as $n_1 := (1+\mu)n_0$. We start Alg afresh for the items with indices $n_0 + 1, n_0 + 2, \dots, \min\{n_1, n\}$. If $n > (1+\mu)n_0$, then we set the new guess of n to be $n_2 := (1+\mu)n_1 = (1+\mu)^2 n_0$ and start Alg afresh on the items with indices $n_1 + 1, n_1 + 2, \dots, \min\{n_2, n\}$. We continue this process of multiplying our estimate of n with $(1+\mu)$ until all the items arrive. See Figure 1 for an illustration. The pseudocode is provided in Algorithm 4.

We consider the following partition of the entire input into super-stages as follows: The first super-stage, Γ_0 , contains the first n_0 items. The second super-stage, Γ_1 , contains the next $n_1 - n_0$ items. In general, for $i > 0$, the $(i+1)$ th super-stage, Γ_i , contains $\min\{n_i, n\} - n_{i-1}$ items which are

Algorithm 4: ImpAlg: Improving Alg to Get Rid of the Assumption on the Knowledge of the Number of Items

```

1: Input:  $I_n(\mathcal{D}) = \{x_1, x_2, \dots, x_n\}$ .
2:  $n_{-1} \leftarrow 0; n_0 \leftarrow \frac{1}{\delta^3}; \text{for } j \geq 1, n_j = (1 + \mu)n_{j-1}$ 
3:  $i \leftarrow 0$ 
4: while true do
5:   Run Alg  $(x_{n_{i-1}+1}, x_{n_{i-1}+2}, \dots, x_{\min\{n, n_i\}})$  with one change that instead of packing the small items in the  $S$ -slots
   created for a stage, we maintain a global set of  $S$ -slots.
6:   if the input stream has ended then
7:     return the packing
8:   else
9:      $i \leftarrow i + 1$ 
10:  end if
11: end while

```

given by $x_{n_{i-1}+1}, x_{n_{i-1}+2}, \dots, x_{\min\{n, n_i\}}$. So, essentially, ImpAlg can be thought of running Alg on each super-stage separately. The number of super-stages is given by $\kappa := \lceil \log_{(1+\mu)}(n/n_0) \rceil$.

Note that $|\Gamma_0| = n_0$, $|\Gamma_1| = \mu n_0$, $|\Gamma_2| = \mu(1 + \mu)n_0$, and so on. In general, for $j \in [\kappa - 2]$, $|\Gamma_j| = \mu(1 + \mu)^{j-1}n_0$. Now consider the last super-stage $\Gamma_{\kappa-1}$ and note that it may not be full, i.e., it can be the case that $|\Gamma_{\kappa-1}| < \mu(1 + \mu)^{\kappa-2}n_0$. So, Alg may pack the last super-stage inefficiently. However, note that $|\Gamma_{\kappa-1}|$ can be at most $\mu(1 + \mu)^{\kappa-2}n_0 \leq \mu n$. Hence the last super-stage contains only a tiny fraction of the input and so, this will have very little effect on the packing of the entire input.

3.3.1 Analysis. When n was known we only had $O(1)$ number of stages. However, now we have $\kappa = \lceil \log_{(1+\mu)}(n/n_0) \rceil$ number of super-stages. There can arise two problems:

- Recall that when n was known, we derived a performance guarantee for each stage individually, that holds w.h.p., and used a union bound to combine all the stages. However, here, since the number of super-stages is a super-constant, we cannot hope for high probability guarantees using a union bound. So, we consider the first $\kappa_1 := \lceil \log_{(1+\mu)}(\delta^7 n) \rceil$ number of the super-stages at a time. We show that these initial super-stages contain only a small fraction of the entire input. Each of the final $(\kappa - \kappa_1)$ super-stages can be individually analyzed using the analysis of Alg.
- For each super-stage, we can have a constant number of S -bins (bins which contain only small items) with less occupancy. However, since the number of super-stages itself is a super-constant, this can result in a lot of wasted space. For this, we exploit the monotonicity of NF to ensure that we can pack small items from a super-stage into empty slots for small items from the previous stages.

We will now proceed to analyze ImpAlg. Recall that the number of super-stages is given by $\kappa = \lceil \log_{(1+\mu)}(n/n_0) \rceil$ where n_0 was defined to be $1/\delta^3$. We will split the analysis into two parts. First, we will analyze the number of bins used by our algorithm in the first $\kappa_1 = \lceil \log_{(1+\mu)}(\delta^7 n) \rceil$ super-stages as a whole. Then, we will analyze the final $\kappa_2 := \kappa - \kappa_1$ super-stages considering each one at a time. We call the first κ_1 super-stages as *initial super-stages* and the remaining κ_2 super-stages as *final super-stages*.

Analysis of the Initial Super-Stages. The basic intuition of the analysis of our algorithm in the initial super-stages is as follows: Since only a small fraction of the entire input is present in these κ_1 super-stages, our algorithm uses only a small fraction of bins compared to the optimal packing

of the entire input. So, we bound the number of large items and the weight of small items and thus bound the number of bins used.

LEMMA 3.7. *Let $\mathcal{A}$ be an algorithm for online BP such that in the packing output by $\mathcal{A}$, every bin, except possibly a constant number of bins τ , which contains only small items has an occupancy of at least $(1 - \delta)$. Let I be a sequence of n i.i.d. items and let J be any contiguous subsequence of I having size βn ($0 < \beta \leq 1$). Suppose $\mathcal{A}$ works in a way such that it packs at most v copies of any large item where v is a constant.⁴ Then the following inequality holds with high probability:*

$$\mathcal{A}(J) \leq \frac{2\beta v}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]).$$

PROOF. Let J_ℓ, I_ℓ respectively denote the set of large items in J, I and let J_s, I_s respectively denote the set of small items in J, I . We prove the lemma considering two cases.

First, we consider the case when $|J_\ell| \leq \delta \text{Opt}(J)$. The number of bins in the packing of J by $\mathcal{A}$ which contain at least one large item is upper bounded by $v|J_\ell|$. The number of bins which contain only small items is upper bounded by $\mathcal{W}(J_s)/(1 - \delta) + \tau$. Hence,

$$\begin{aligned} \mathcal{A}(J) &\leq v|J_\ell| + \frac{\mathcal{W}(J_s)}{1-\delta} + \tau \\ &\leq v\delta \text{Opt}(J) + \frac{\text{Opt}(J)}{1-\delta} + \tau \\ &= \frac{1 + v(1-\delta)\delta}{1-\delta} \text{Opt}(J) + \tau \\ &\leq 2v(1+\delta) \text{Opt}(J) + \tau && (\text{since } 0 < \delta < 1/2 \text{ and } v \geq 1) \\ &\leq 2\beta v(1+\delta)(1+2\delta) \mathbb{E}[\text{Opt}(I)] + C_0 \mathbb{E}[\text{Opt}(I)]^{2/3} && (\text{w.h.p. for some constant } C_0) \\ & && (\text{using Lemma 3.6}) \\ &\leq \frac{2\beta v}{\delta(1-\delta)} \mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)]) && (\text{since } \delta < 1/2, 1+\delta < \frac{1}{1-\delta} \text{ and } 1+2\delta < \frac{1}{\delta}) \end{aligned}$$

Next, we consider the case when $|J_\ell| > \delta \text{Opt}(J)$. We will again bound the quantity $v|J_\ell| + \frac{\mathcal{W}(J_s)}{1-\delta} + \tau$. To bound $|J_\ell|$, we use Bernstein's inequality (Lemma 3.2). Let $X_1, X_2, \dots, X_{|J|}$ be random variables where X_i takes value 1 if the i th item in J is large and 0 otherwise. Then, clearly, $|J_\ell| = \sum_{i=1}^{|J|} X_i$. Also, note that $\mathbb{E}[|J_\ell|] = \beta \mathbb{E}[|I_\ell|]$. Moreover, we can derive the following inequalities that hold with high probability:

$$\begin{aligned} \mathbb{E}[|I_\ell|] &\geq \frac{1}{\beta} |J_\ell| - \frac{1}{\beta} \mathbb{E}[\mathcal{W}(I)]^{2/3} && (\text{using Lemma 3.5}) \\ &\geq \frac{\delta}{\beta} \text{Opt}(J) - \frac{1}{\beta} \mathbb{E}[\mathcal{W}(I)]^{2/3} \\ &\geq \frac{\delta}{\beta} \mathcal{W}(J) - \frac{1}{\beta} \mathbb{E}[\mathcal{W}(I)]^{2/3} \\ &\geq \frac{\delta}{\beta} \left(\beta \mathbb{E}[\mathcal{W}(I)] - \mathbb{E}[\mathcal{W}(I)]^{2/3} \right) - \frac{1}{\beta} \mathbb{E}[\mathcal{W}(I)]^{2/3} && (\text{using Lemma 3.4}) \end{aligned}$$

⁴For example, **A1g** does this. For each stage, it computes the proxy packing of the previous stage. Hence, every large item is potentially packed twice. In the worst case, these copies may not get replaced, thus resulting in wasted space.

$$\geq \frac{\delta}{2} \mathbb{E} [\text{Opt}(I)] - o(\mathbb{E} [\text{Opt}(I)]). \quad (\text{since } \mathcal{W}(I) \leq \text{Opt}(I) \leq 2\mathcal{W}(I))$$

Hence, from now on, we will assume that there exists a constant $a_1 > 0$ such that

$$\mathbb{E} [|I_\ell|] \geq a_1 \mathbb{E} [\text{Opt}(I)] \quad (3.31)$$

holds with high probability. Using Bernstein's inequality (Lemma 3.2),

$$\begin{aligned} \mathbb{P} [|J_\ell| \geq 2\beta \mathbb{E} [|I_\ell|]] &= \mathbb{P} [|J_\ell| \geq \mathbb{E} [|J_\ell|] + \beta \mathbb{E} [|I_\ell|]] \\ &\leq 2 \exp \left(-\frac{\beta^2 \mathbb{E} [|I_\ell|]^2}{2\mathbb{E} [|J_\ell|] + \frac{2}{3}\beta \mathbb{E} [|I_\ell|]} \right) \\ &= 2 \exp \left(-\frac{3}{8} \beta \mathbb{E} [|I_\ell|] \right) \\ &\leq 2 \exp \left(-\frac{3}{8} \beta a_1 \cdot \mathbb{E} [\text{Opt}(I)] \right). \quad (\text{from Equation (3.31)}) \end{aligned}$$

Now to bound $\mathcal{W}(J_s)$, we will again use Bernstein's inequality on a new set of random variables $X_1, X_2, \dots, X_{|J|}$ where X_i equals the weight of the i th item in J if it is small and 0 otherwise. Clearly, $\sum_{i=1}^{|J|} X_i = \mathcal{W}(J_s)$ and $\mathbb{E} [\mathcal{W}(J_s)] = \beta \mathbb{E} [\mathcal{W}(I_s)] \leq \beta \mathbb{E} [\mathcal{W}(I)]$. Thus,

$$\begin{aligned} \mathbb{P} [\mathcal{W}(J_s) \geq 2\beta \mathbb{E} [\mathcal{W}(I)]] &\leq \mathbb{P} [\mathcal{W}(J_s) \geq \mathbb{E} [\mathcal{W}(J_s)] + \beta \mathbb{E} [\mathcal{W}(I)]] \\ &\leq 2 \exp \left(\frac{-\beta^2 \mathbb{E} [\mathcal{W}(I)]^2}{2\mathbb{E} [\mathcal{W}(J_s)] + \frac{2}{3}\beta \mathbb{E} [\mathcal{W}(I)]} \right) \\ &= 2 \exp \left(\frac{-3\beta}{8} \mathbb{E} [\mathcal{W}(I)] \right) \\ &\leq 2 \exp \left(\frac{-3\beta \mathbb{E} [\text{Opt}(I)]}{16} \right). \quad (\text{since } 2\mathcal{W}(I) \geq \text{Opt}(I)) \end{aligned}$$

Thus, with high probability, we have that $|J_\ell| \leq 2\beta \mathbb{E} [|I_\ell|]$ and $\mathcal{W}(J_s) \leq 2\beta \mathbb{E} [\mathcal{W}(I)]$. Hence,

$$\begin{aligned} \mathcal{A}(J) &\leq v |J_\ell| + \frac{\mathcal{W}(J_s)}{1-\delta} + \tau \\ &\leq 2v\beta \mathbb{E} [|I_\ell|] + \frac{2\beta \mathbb{E} [\mathcal{W}(I)]}{1-\delta} + \tau \\ &\leq \frac{2v\beta}{\delta} \mathbb{E} [\text{Opt}(I)] + \frac{2\beta \mathbb{E} [\text{Opt}(I)]}{1-\delta} + \tau \\ &= \frac{2v\beta}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \end{aligned}$$

This completes the proof. $\square$

With the help of the above lemma, we proceed to analyze ImpAlg for the initial (first κ_1) super-stages. Note that the number of items in the initial super-stages is given by $(1+\varepsilon)^{\kappa_1-1} n_0 \leq (\delta^7 n) n_0 = \delta^4 n$. Since Alg satisfies the properties of $\mathcal{A}$ in Lemma 3.7 and ImpAlg just applies Alg multiple times, we can use Lemma 3.7 to analyze ImpAlg.

There is one difficulty though. Let's call a bin which contains only small items to be an S-bin. Although in a super-stage the number of S-bins not filled up to a level of at least $(1-\delta)$ is a constant, the number of initial super-stages itself is not a constant. We can work around this problem by continuing NF to pack the small items in the S-slots created during the previous stages and the

previous super-stages as well. In other words, instead of packing the small items of a stage in S -slots created only during that stage, we keep a global set of S -slots and pack the small items in them using NF. We now have to show that our algorithm doesn't increase the number of bins with this change. Since the way in which the large items are packed hasn't changed, the number of bins that contain large items does not change. Since NF is monotone (even with varying bin sizes, [40]), the number of bins containing only the small items will either decrease or stays the same.

Using Lemma 3.7 with $\beta = \delta^4$ and $\nu = 2$ (since in each super-stage, a large item is packed at most twice—once when it arrives as a real item, and once when it is used as a proxy item), we obtain that with high probability,

$$\begin{aligned} \text{Alg}(\Gamma_0) + \text{Alg}(\Gamma_1) + \cdots + \text{Alg}(\Gamma_{\kappa_1-1}) &\leq \frac{2(2)\delta^4}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\ &\leq 8\delta^3 \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\ &\leq 8\delta \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \end{aligned} \quad (3.32)$$

Analysis of the Final Super-stages. For this part, the important thing to note is that $\kappa_2 \leq \lceil \log_{(1+\mu)}(1/(\delta^7 n_0)) \rceil$ is a constant. Moreover, since the number of items in the initial super-stages is at least $\delta^4 n / (1 + \mu)$, each of the final super-stages has at least $\mu \delta^4 n / (1 + \mu)$ items (which tends to infinity in the limiting case). Thus, we can use the analysis of Alg for each of these final super-stages. As mentioned, since the last super-stage might not be full, we analyze the last super-stage differently. All the other super-stages are full. So, we can directly use the analysis of Alg. For all $\kappa_1 \leq i < \kappa - 1$, from the analysis of Alg (Equation (3.29)), we have that with high probability

$$\text{Alg}(\Gamma_i) \leq \alpha(1 + 8\delta) \mathbb{E} [\text{Opt}(\Gamma_i)] + C \mathbb{E} [\text{Opt}(\Gamma_i)]^{2/3} + o(\text{Opt}(\Gamma_i))$$

for some constant C .

Now, to bound $\text{Opt}(\Gamma_i)$ in terms of $\text{Opt}(I)$, we can use Lemma 3.6. Thus, the above inequality transforms into

$$\begin{aligned} \text{Alg}(\Gamma_i) &\leq \alpha(1 + 8\delta)(1 + 2\delta) \frac{|\Gamma_i|}{n} \mathbb{E} [\text{Opt}(I)] + C_1 \mathbb{E} [\text{Opt}(\Gamma_i)]^{2/3} + o(\text{Opt}(\Gamma_i)) \\ &\leq \alpha(1 + 8\delta)(1 + 2\delta) \frac{|\Gamma_i|}{n} \mathbb{E} [\text{Opt}(I)] + C_1 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\text{Opt}(I)) \end{aligned}$$

for some constant C_1 . Summing over all i ($\kappa_1 \leq i < \kappa - 1$), and observing that $\kappa - \kappa_1$ is a constant, we obtain that with high probability,

$$\begin{aligned} &\text{Alg}(\Gamma_{\kappa_1}) + \text{Alg}(\Gamma_{\kappa_1+1}) + \cdots + \text{Alg}(\Gamma_{\kappa-2}) \\ &\leq \alpha(1 + 8\delta)(1 + 2\delta) \sum_{i=\kappa_1}^{\kappa-2} \frac{|\Gamma_i|}{n} \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\text{Opt}(I)) \\ &\leq \alpha(1 + 26\delta) \sum_{i=\kappa_1}^{\kappa-2} \frac{|\Gamma_i|}{n} \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\text{Opt}(I)) \\ &\leq \alpha(1 + 26\delta) \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\text{Opt}(I)) \end{aligned} \quad (3.33)$$

for some constant C_2 .

Now consider the last super-stage $\Gamma_{\kappa-1}$. If $|\Gamma_{\kappa-1}|$ is exactly equal to $n_{\kappa-1} - n_{\kappa-2}$, then we can obtain the same bound as Equation (3.33). However, this might not be the case. The last super-stage

contains $n - n_{\kappa-2}$ items.

$$\begin{aligned} n - n_{\kappa-2} &\leq n_{\kappa-1} - n_{\kappa-2} = (1 + \mu)^{\kappa-1} n_0 - (1 + \mu)^{\kappa-2} n_0 = \mu(1 + \mu)^{\kappa-2} n_0 \\ &= \mu n_{\kappa-2} \\ &\leq \mu n. \end{aligned}$$

Hence, the size of the last super-stage is at most μ fraction of the entire input size. We will again use Lemma 3.7 with $\beta = \mu$ and $\nu = 2$ for the last super-stage. We thus obtain that

$$\begin{aligned} \text{Alg}(\Gamma_{\kappa-1}) &\leq \frac{(2)(2)(|\Gamma_{\kappa-1}|/n)}{n\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\ &\leq \frac{4\mu}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \\ &\leq 8\delta \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \end{aligned} \quad (3.34)$$

with high probability. The last inequality follows since $\mu = \delta^2$ and $\delta < 1/2$.

Summing Equations (3.32) to (3.34), we obtain that with high probability, for some constant C_4 ,

$$\begin{aligned} \text{ImpAlg}(I) &= \sum_{j=0}^{\kappa_1-1} \text{Alg}(\Gamma_j) + \sum_{j=\kappa_1}^{\kappa-2} \text{Alg}(\Gamma_j) + \text{Alg}(\Gamma_{\kappa-1}) \\ &\leq (\alpha(1 + 26\delta) + 16\delta) \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\mathbb{E} [\text{Opt}(I)]) + o(\text{Opt}(I)) \\ &\leq \alpha(1 + 42\delta) \mathbb{E} [\text{Opt}(I)] + C_2 \mathbb{E} [\text{Opt}(I)]^{2/3} + o(\mathbb{E} [\text{Opt}(I)]) + o(\text{Opt}(I)). \end{aligned} \quad (3.35)$$

Equation (3.35) holds with high probability, say $p = 1 - o(1)$. In the scenario where the low probability event occurs, we can bound the number of bins used by ImpAlg using Lemma 3.7 with $\beta = 1$ and $\nu = 2$:

$$\text{ImpAlg}(I) \leq \frac{2 \cdot 2}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]).$$

Let E be the event when Equation (3.35) holds. Then

$$\begin{aligned} \mathbb{E} [\text{ImpAlg}(I)] &= \mathbb{E} [\text{ImpAlg}(I)|E] \mathbb{P} [E] + \mathbb{E} [\text{ImpAlg}(I)|\bar{E}] \mathbb{P} [\bar{E}] \\ &\leq \left((\alpha(1 + 42\delta)) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]) \right) (1 - o(1)) \\ &\quad + \left(\frac{4}{\delta(1-\delta)} \mathbb{E} [\text{Opt}(I)] + 1 \right) o(1) \\ &= (\alpha(1 + 42\delta)) \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]). \end{aligned}$$

Choosing $\delta = \frac{\epsilon}{42\alpha}$ ensures that the CR of ImpAlg is $\alpha + \epsilon$.

4 BF under the Random-Order Model

In this section, we will prove Theorems 1.3 and 1.4. To recall, Theorem 1.4 shows that BF achieves a random-order ratio of exactly 1 when all the item sizes are more than $1/3$, while Theorem 1.3 shows a random-order ratio of 1.49107 when the item sizes are in the range $(1/4, 1/2]$.

4.1 Performance of BF When Item Sizes Are Larger Than $1/3$

Albers et al. [1] showed that the asymptotic random-order ratio of the BF algorithm is at most 1.25 when all the item sizes are more than $1/3$. In this section, we improve it further and show that, BF for this special case under the random-order model is nearly optimal. We will use the upright matching result given by Lemma 2.2. We restate the lemma again for convenience.

LEMMA 2.2 (UPRIGHT MATCHING WITH RANDOMLY PERMUTED COORDINATES [10]). *Consider a set of reals $r_1, r_2, \dots, r_m$ and $s_1, s_2, \dots, s_m$ such that $r_i \leq s_i$ for all $i \in [m]$. Consider a uniform random permutation π of $[2m]$. Define the set of minus points $P^- = \{(\pi(i), r_i)\}_{i \in [m]}$, and the set of plus points $P^+ = \{(\pi(m+i), s_i)\}_{i \in [m]}$. Then, there exist universal constants a, C, K such that with probability at least $1 - C \exp(-a(\log m)^{3/2})$, we have that*

$$U(P^+, P^-) \leq K\sqrt{m}(\log m)^{3/4}.$$

Remark 1. The above lemma can be looked at as follows. Consider an upright matching instance where the minus points are given by the set $P^- = \{(i, r_i)\}_{i \in [m]}$ and the plus points are given by the set $P^+ = \{(m+i, s_i)\}_{i \in [m]}$. Now, if we randomly permute the x-coordinates of the points in $P^+ \cup P^-$, then the number of unmatched points in a maximum upright matching is very low.

We now proceed to prove Theorem 1.3. We first show that the **modified BF (MBF)** algorithm [12] is nearly optimal using Lemma 2.2. The MBF algorithm is the same as BF except that it closes a bin if it receives an item of size less than $1/2$. Hence, note that any bin packed by MBF can have at most two items. Shor [47] showed that MBF *dominates* BF, i.e., for any instance I , $\text{BF}(I) \leq \text{MBF}(I)$. MBF can be easily reduced to upright matching as follows. Consider any BP instance $I = \{x_1, x_2, \dots, x_n\}$ and an arbitrary item $x_i \in I$.

- If $x_i \leq 1/2$, then we create a plus point $(i, 1 - x_i)$.
- If $x_i \geq 1/2$, then we create a minus point (i, x_i) .

Now, it can be seen that the maximum upright matching algorithm given by Algorithm 1 for the above instance exactly corresponds to the MBF algorithm on I . This is because any plus point p^+ corresponds to an item x of size at most $1/2$ and any minus point p^- corresponds to an item y of size more than $1/2$, and p^+ is matched to p^- by Algorithm 1 iff the item x is packed on top of y by MBF.

Call an item x large (L) if $x > 1/2$ and medium (M) if $x \in (1/3, 1/2]$. We define a bin as LM -bin if it contains one large item and one medium item. We use the following lemma which was proved in [1] using the monotonicity property of BF when all item sizes are more than $1/3$.

LEMMA 4.1 [1]. *Consider the set of BP instances $\mathcal{J}$ where each instance $J \in \mathcal{J}$ only has large and medium items, and every bin in $\text{Opt}(J)$ is an LM -bin. If BF has an AAR of α when restricted to the instances in $\mathcal{J}$, then it has an AAR of α for any list of items larger than $1/3$ as well.*

Consider an input instance which has an optimal packing containing only LM -bins. Consider the number of bins opened by MBF for such instances. Each large item definitely opens a new bin, and a medium item opens a new bin if and only if it cannot be placed along with a large item, i.e., it is “unmatched.” So, the number of bins opened by MBF equals (number of large items + number of unmatched medium items). Now, we will prove our result.

THEOREM 4.2. *For any list I of items larger than $1/3$, the $\text{ARR } \text{RR}_{\text{BF}}^\infty = 1$.*

PROOF. From Lemma 4.1, it is enough to prove the theorem for any list I for which $\text{Opt}(I)$ is only made up of LM -bins. So, we can assume that I has k large items and k medium items where $\text{Opt}(I) = k$. Now consider the packing of MBF for a randomly permuted list I_σ . We have

$$\text{MBF}(I_\sigma) = (k + \text{number of unmatched medium items}).$$

We have already seen that MBF can be reduced to the maximum upright matching algorithm given by Algorithm 1. This, coupled with Remark 1, tells us that the number of unmatched medium items

is at most $O\left(\sqrt{k} \log^{3/4} k\right)$. So, we have

$$\begin{aligned} \text{MBF}(I_\sigma) &\leq k + O\left(\sqrt{k}(\log k)^{3/4}\right) \\ &= \text{Opt}(I) + o(\text{Opt}(I)) \end{aligned}$$

with probability of at least $1 - C \exp(-a(\log \text{Opt}(I))^{3/2})$ for some universal constants $a, C, K > 0$. Since MBF dominates BF, we have

$$\mathbb{P}[\text{BF}(I_\sigma) \leq \text{Opt}(I) + o(\text{Opt}(I))] \geq 1 - C \exp\left(-a \log^{3/2} \text{Opt}(I)\right).$$

In case the high probability event does not occur, we can use the bound of $\text{BF}(I_\sigma) \leq 1.7\text{Opt}(I) + 2$. Let $p := C \exp(-a(\log \text{Opt}(I))^{3/2})$. Then

$$\begin{aligned} \mathbb{E}[\text{BF}(I_\sigma)] &\leq p(1.7\mathbb{E}[\text{Opt}(I)] + 2) + (1 - p)(\mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)])) \\ &\leq \mathbb{E}[\text{Opt}(I)] + o(\mathbb{E}[\text{Opt}(I)]). \end{aligned} \quad (\text{since } p = o(1))$$

So, we get

$$RR_{BF}^\infty = \limsup_{k \rightarrow \infty} \left(\sup_{I: \text{Opt}(I)=k} (\mathbb{E}[\text{BF}(I_\sigma)] / \text{Opt}(I)) \right) = 1.$$

This completes the proof. $\square$

4.2 The 3-Partition Problem under Random-Order Model

In this section, we analyze the BF algorithm under the random-order model given that the item sizes lie in the range $(1/4, 1/2]$, and thus prove Theorem 1.4. We call an item *small* if its size lies in the range $(1/4, 1/3]$ and *medium* if its size lies in the range $(1/3, 1/2]$. Let I be the input list of items and let $n := |I|$. Recall that given σ , a uniform random permutation of $[n]$, I_σ denotes the list I permuted according to σ . We denote by $\text{Opt}(I_\sigma)$, the number of bins used in the optimal packing of I_σ and by $\text{BF}(I_\sigma)$, the number of bins used by BF to pack I_σ . Note that $\text{Opt}(I_\sigma) = \text{Opt}(I)$.

If there exists a set of three small items in I_σ such that they arrive as three consecutive items, we call that set to be an *S-triplet*. We call a bin to be a *k-bin* if it contains exactly k items, for $k \in \{1, 2, 3\}$. We sometimes refer to a bin by mentioning its contents more specifically as follows: An *MS-bin* is a 2-bin which contains a medium item and a small item. Similarly, an *SSS-bin* is a 3-bin which contains three small items. Likewise, we can define an *M-bin*, *S-bin*, *MM-bin*, *SS-bin*, *MMS-bin*, and *MSS-bin*.

Since the item sizes lie in $(1/4, 1/2]$, any bin in the optimal packing contains at most three items. For the same reason, in the packing by BF, every bin (with one possible exception) contains at least two items. This trivially shows that the ECR of BF is at most $3/2$. To break the barrier of $3/2$, we use the following observations:

- Any 3-bin must contain a small item.
- So, if the optimal solution contains a lot of 3-bins, then it means that the input set contains a lot of small items.

We will prove that if there exist many small items in the input, then with high probability, in a random permutation of the input, there exist many disjoint *S-triplets*.

LEMMA 4.3. *Let m be the number of small items in the input set I , and let X_σ denote the maximum number of mutually disjoint *S-triplets* in I_σ . Suppose $m \geq cn$ where c is a positive constant, then the*

following statements hold true:

- (1) $\mathbb{E}[X_\sigma] \geq m^3/(3n^2) \geq c^3n/3$.
- (2) $X_\sigma \geq c^3n/3 - o(n)$ with high probability.

Then, we prove that BF packs at least one small item from an S -triplet in a 3-bin or in an SS -bin.

LEMMA 4.4. *In the input sequence I_σ , let κ denote the number of disjoint S -triplets. Then, in the BF packing of I_σ , there will be at least $\lfloor \kappa/2 \rfloor$ number of 3-bins.*

But the number of SS -bins in the final packing of BF can be at most one. So, we obtain that the number of 3-bins in the BF packing is significant. With these arguments, the proof of Theorem 1.4 follows. Now, we give the detailed proofs of the above two lemmas now.

PROOF OF LEMMA 4.3. One can construct a random permutation of the input list I by first placing the small items in a random order and then inserting the remaining items among the small items randomly.

After placing the small items, we have $m + 1$ gaps to place the remaining items as shown below in the form of empty squares.

$$\underbrace{\square S \square S \square S}_{\text{Triplet } T_1} \underbrace{\square S \square S \square S}_{\text{Triplet } T_2} \underbrace{\square S \square S \square S}_{\text{Triplet } T_3} \cdots \underbrace{\square S \square S \square S}_{\text{Triplet } T_{m/3}} \square$$

As shown above, we name the S -triplets as $T_1, T_2, \dots, T_{m/3}$. (Strictly speaking, the number of S -triplets should be $\lfloor m/3 \rfloor$, but we relax this since we are only interested in the asymptotic case and $m \geq cn$.) Let us start inserting the medium items into these empty squares. Consider any S -triplet T_i . The probability that T_i continues to be an S -triplet after inserting the first medium item is $(m-1)/(m+1)$. This is because among the $m+1$ squares, we have $m-1$ choices of squares for the first medium item to be inserted in. The first medium item thus occupies one of the squares, but in this process, it creates two new squares on either side of itself, thereby increasing the net number of squares by one. Hence, the probability that T_i continues to be an S -triplet after inserting the second medium item (conditioned on it being a consecutive S -triplet after inserting the first medium item) is $m/(m+2)$. We continue this process of inserting the medium items and after they all have been inserted, let $Y_\sigma^{(i)}$ be the indicator random variable denoting whether T_i is consecutive or not. Then

$$\mathbb{E}[Y_\sigma^{(i)}] = \frac{m-1}{m+1} \frac{m}{m+2} \cdots \frac{n-2}{n} = \frac{m(m-1)}{n(n-1)}.$$

Let $Y_\sigma = Y_\sigma^{(1)} + Y_\sigma^{(2)} + \cdots + Y_\sigma^{(m/3)}$. It can be seen that Y_σ is a lower bound on X_σ which was defined as the maximum number of mutually disjoint S -triplets in I_σ in the lemma statement. By linearity of expectations,

$$\lim_{n, m \rightarrow \infty} \mathbb{E}[Y_\sigma] = \lim_{n, m \rightarrow \infty} \frac{m}{3} \frac{m(m-1)}{n(n-1)} \approx \frac{m^3}{3n^2} \geq \frac{c^3n}{3}.$$

Since $X_\sigma \geq Y_\sigma$, the first part of the lemma follows.

To prove the second part of the lemma, we will compute $\text{Var}[Y_\sigma]$ and use Chebyshev's inequality to show that Y_σ is concentrated around its mean. For any i , note that

$$\text{Var}[Y_\sigma^{(i)}] = \mathbb{E}[Y_\sigma^{(i)}] - \mathbb{E}[Y_\sigma^{(i)}]^2 < 1. \quad (4.1)$$

This is because $Y_\sigma^{(i)}$ has expectation at most 1.

Since Y_σ is a sum of $Y_\sigma^{(i)}$ -s, to calculate the variance of Y_σ , we also need to calculate the covariance of each pair of random variables in the set $\{Y_\sigma^{(1)}, Y_\sigma^{(2)}, \dots, Y_\sigma^{(m/3)}\}$. Intuitively, it is clear that for $j \neq k$, the random variables $Y_\sigma^{(j)}$ and $Y_\sigma^{(k)}$ are negatively correlated. This is because if the S -triplet T_j remains to be an S -triplet after inserting all the small items, then the probability of the medium items being inserted in between the gaps of the small items in the S -triplet T_k increase. Hence, the chances that the S -triplet T_k remains to be an S -triplet decrease. We formalize this intuition now.

Consider any two S -triplets T_j, T_k where $j, k \in [m/3]$ and $j \neq k$. The probability that both T_j and T_k remain to be S -triplets after inserting the first medium item is $(m-3)/(m+1)$. This is because, among the $m+1$ squares, the first medium item can be inserted in $m-3$ squares in order to ensure that the S -triplets T_j, T_k continue to be S -triplets. Similarly, the probability that both T_j and T_k remain to be S -triplets after inserting the second medium item is $(m-2)/(m+2)$. Continuing in this manner, after all the medium items have been inserted,

$$\mathbb{E} \left[Y_\sigma^{(j)} Y_\sigma^{(k)} \right] = \mathbb{P} \left[Y_\sigma^{(j)} = 1 \wedge Y_\sigma^{(k)} = 1 \right] = \frac{m-3}{m+1} \frac{m-2}{m+2} \dots \frac{n-4}{n} = \frac{m(m-1)(m-2)(m-3)}{n(n-1)(n-2)(n-3)}.$$

The covariance of $Y_\sigma^{(j)}, Y_\sigma^{(k)}$ is given by

$$\begin{aligned} \text{Cov} \left[Y_\sigma^{(j)}, Y_\sigma^{(k)} \right] &= \mathbb{E} \left[Y_\sigma^{(j)} Y_\sigma^{(k)} \right] - \mathbb{E} \left[Y_\sigma^{(j)} \right] \mathbb{E} \left[Y_\sigma^{(k)} \right] \\ &= \frac{m(m-1)(m-2)(m-3)}{n(n-1)(n-2)(n-3)} - \frac{m^2(m-1)^2}{n^2(n-1)^2} \\ &= \frac{m(m-1)}{n(n-1)} \left(\frac{(m-2)(m-3)}{(n-2)(n-3)} - \frac{m(m-1)}{n(n-1)} \right). \end{aligned}$$

For any reals x, y such that $2 < x < y$, we have the identity $\frac{x-2}{y-2} < \frac{x}{y}$. Hence, we obtain that

$$\text{Cov} \left[Y_\sigma^{(j)}, Y_\sigma^{(k)} \right] < 0. \quad (4.2)$$

Combining all these, the variance of Y_σ can be calculated as follows:

$$\begin{aligned} \text{Var} [Y_\sigma] &= \sum_{i=1}^{m/3} \text{Var} \left[Y_\sigma^{(i)} \right] + 2 \sum_{1 \leq j < k \leq m/3} \text{Cov} \left[Y_\sigma^{(j)}, Y_\sigma^{(k)} \right] \\ &\leq \frac{m}{3} \quad (\text{from Equations (4.1) and (4.2)}) \\ &\leq n. \end{aligned}$$

Now using Chebyshev's inequality,

$$\begin{aligned} \mathbb{P} \left[Y_\sigma \leq \mathbb{E} [Y_\sigma] - (\mathbb{E} [Y_\sigma])^{2/3} \right] &\leq \mathbb{P} \left[|Y_\sigma - \mathbb{E} [Y_\sigma]| \geq (\mathbb{E} [Y_\sigma])^{2/3} \right] \\ &\leq \frac{\text{Var} [Y_\sigma]}{(\mathbb{E} [Y_\sigma])^{4/3}} \\ &\leq \frac{n}{\frac{c^4}{3^{4/3}} n^{4/3}} \\ &= O \left(\frac{1}{n^{1/3}} \right), \text{ as } c \text{ is a constant.} \end{aligned}$$

Since X_σ is the maximum number of disjoint S -triplets and Y_σ denotes the number of disjoint S -triplets among $T_1, T_2, \dots, T_{m/3}$ after inserting all the medium items, we have $X_\sigma \geq Y_\sigma \geq c^3 n/3 - o(n)$ with high probability. $\square$

PROOF OF LEMMA 4.4. To prove the lemma, we will show that each S -triplet will result in the formation of a 3-bin or an SS -bin. A property of BF is that, at any point in time, there can be at most one bin of load at most $1/2$. An SS -bin has a load at most $1/2$; hence, every SS -bin (with at most one exception) created by the S -triplets will transform into a 3-bin.

Let $\{S_1, S_2, S_3\}$ be an S -triplet in I_σ such that S_3 follows S_2 which in turn follows S_1 . We consider three cases to show that this S -triplet will result in the formation of a 3-bin or an SS -bin.

- If S_1 is going to be packed in a 2-bin. Then after packing S_1 , this bin becomes a 3-bin and hence the lemma holds.
- Suppose S_1 is going to be packed in a 1-bin B . Then just before the arrival of S_1 , B must have been the only 1-bin and hence, after packing S_1 , each bin is either a 2-bin or a 3-bin.
 - Now, if one of S_2, S_3 is packed into a 2-bin, then the bin becomes a 3-bin and the lemma follows.
 - Otherwise, both S_2, S_3 are packed into a single new SS -bin and hence the lemma follows.
- Suppose S_1 is packed in a new bin. Then just prior to the arrival of S_2 , every bin is either a 2-bin or 3-bin except the bin containing S_1 . Thus, S_2 is either packed in a 2-bin (thus becoming a 3-bin) or packed in the bin containing S_1 , resulting in an SS -bin.

Thus, we have shown that each S -triplet will result in the formation of a 3-bin or an SS -bin. At any point of time, the number of SS -bins can be at most one. Therefore, every SS -bin (except at most one) created by the S -triplets will end up as a 3-bin due to the future items.

However, consider the following scenario where an S -triplet $\{S_1, S_2, S_3\}$ creates an SS -bin B_1 and a future S -triplet results in the formation of a 3-bin B_2 , and $B_1 = B_2$. In case such a scenario occurs, two S -triplets correspond to only one 3-bin in the packing of BF. But once this scenario occurs, the bin $B_1 (= B_2)$ will be closed, i.e., no more items will be packed in it.

Hence, we obtain that if there are κ number of S -triplets in the input sequence I_σ , there will be at least $\lfloor \kappa/2 \rfloor$ number of 3-bins in the BF packing of I_σ . $\square$

Now, using Lemmas 4.3 and 4.4, we prove Theorem 1.4.

PROOF OF THEOREM 1.4. Let X_3 be the number of 3-bins in the final BF packing of I_σ . The remaining $(n - 3X_3)$ items are packed in 2-bins and at most one 1-bin. Therefore,

$$\text{BF}(I_\sigma) \leq X_3 + \frac{n - 3X_3}{2} + 1 = \frac{n - X_3}{2} + 1.$$

Since any bin in the optimal solution can accommodate at most three items, we have that $\text{Opt}(I) \geq n/3$. Hence,

$$\text{BF}(I_\sigma) \leq \frac{3}{2} \left(1 - \frac{X_3}{n} \right) \text{Opt}(I) + 1. \quad (4.3)$$

Let $z_1 (\leq 1)$, z_2 , and z_3 be the number of 1-bins, 2-bins, and 3-bins in the optimal packing of I_σ , respectively. Then, $\text{Opt}(I) = z_1 + z_2 + z_3$ and $n = z_1 + 2z_2 + 3z_3$. Note that any two items can fit in a bin. Define the quantity $\mu := z_2/\text{Opt}(I)$, the fraction of 2-bins in the optimal solution. We obtain

$$\text{BF}(I_\sigma) \leq \frac{n + 1}{2} = \frac{z_1 + 2z_2 + 3z_3 + 1}{2} \leq \frac{3\text{Opt}(I) - z_2 - 2z_1 + 1}{2} \leq \left(\frac{3}{2} - \frac{\mu}{2} \right) \text{Opt}(I) + 1. \quad (4.4)$$

When μ is close to one we already obtain a CR very close to 1 due to the above inequality. When μ is significantly less than 1, we analyze the BF packing of I_σ in a different way. Let n_s be the number of small items. Since a 3-bin contains at least one small item, we know that, $n_s \geq z_3 = \text{Opt}(I) - z_2 - z_1 = \text{Opt}(I) - \mu \text{Opt}(I) - z_1 = (1 - \mu) \text{Opt}(I) - z_1$. On the other hand, we have that $n = z_1 + 2z_2 + 3z_3 = \text{Opt}(I) + z_2 + 2z_3 = \text{Opt}(I) + \mu \text{Opt}(I) + 2((1 - \mu) \text{Opt}(I) - 1) \leq (3 - \mu) \text{Opt}(I)$. Thus, we have following two inequalities:

$$n_s \geq (1 - \mu) \text{Opt}(I) - z_1 \quad \text{and} \quad n \leq (3 - \mu) \text{Opt}(I).$$

Hence, the fraction of small items in the input sequence is given by

$$f_s := \frac{n_s}{n} \geq \frac{(1 - \mu) \text{Opt}(I) - z_1}{(3 - \mu) \text{Opt}(I)}.$$

Let X_σ be the random variable which denotes the maximum number of S-triplets in the input sequence I_σ . By Lemma 4.3,

$$X_\sigma \geq \frac{((1 - \mu) \text{Opt}(I_\sigma) - z_1)^3}{3(3 - \mu)^3 \text{Opt}(I)^3} n - o(n) = \frac{(1 - \mu)^3}{3(3 - \mu)^3} n - o(n) \text{ w.h.p., as } z_1 \leq 1.$$

Recall that X_3 denotes the number of 3-bins in the BF packing of I_σ . By Lemma 4.4,

$$X_3 \geq \frac{X_\sigma}{2} - 1 \geq \frac{(1 - \mu)^3}{6(3 - \mu)^3} n - o(n) \quad (4.5)$$

with high probability. Substituting Equation (4.5) in Equation (4.3), we get

$$\begin{aligned} \text{BF}(I_\sigma) &\leq \frac{3}{2} \left(1 - \frac{o(n)}{n} - \frac{(1 - \mu)^3}{6(3 - \mu)^3} \right) \text{Opt}(I_\sigma) + 1 \\ &\leq \frac{3}{2} \left(1 - \frac{(1 - \mu)^3}{6(3 - \mu)^3} \right) \text{Opt}(I_\sigma) + o(\text{Opt}(I_\sigma)) \end{aligned} \quad (4.6)$$

with high probability. Equations (4.4) and (4.6) are the result of two different ways of analyzing the same algorithm. Hence, the ECR is at most

$$\begin{aligned} &\min \left\{ \frac{3}{2} - \frac{\mu}{2}, \frac{3}{2} - \frac{(1 - \mu)^3}{4(3 - \mu)^3} \right\} \\ &= \frac{3}{2} - \frac{1}{2} \max \left\{ \mu, \frac{(1 - \mu)^3}{2(3 - \mu)^3} \right\}. \end{aligned}$$

In the domain $(0, 1)$, the function μ is increasing and takes value zero at $\mu = 0$, while the function $\frac{1 - \mu}{3 - \mu}$ is decreasing and takes value zero at $\mu = 1$. Hence, the maximum among both the functions is achieved when both the functions are equal, i.e., for a $\mu \in [0, 1]$ satisfying

$$\mu = \frac{(1 - \mu)^3}{2(3 - \mu)^3}.$$

This happens when $\mu = 0.017861$ (can be verified by substitution); the approximation ratio, in this case, is roughly 1.49107.

To summarize, we proved that, with high probability, say $p = 1 - o(1)$, $\text{BF}(I_\sigma) \leq 1.49107 \text{Opt}(I) + o(\text{Opt}(I))$. The fact that $\text{BF}(I_\sigma) \leq 1.7 \text{Opt}(I) + 2$ always holds due to [30]. Combining these, we get that $\mathbb{E} [\text{BF}(I)] \leq p(1.49107 \cdot \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)])) + (1 - p)(1.7 \mathbb{E} [\text{Opt}(I)] + 2)$. Thus we obtain that

$$\mathbb{E} [\text{Opt}(I)] \leq 1.49107 \cdot \mathbb{E} [\text{Opt}(I)] + o(\mathbb{E} [\text{Opt}(I)]).$$

This completes the proof of Theorem 1.4. □

5 Conclusion

We studied online BP under two stochastic settings, namely the i.i.d. model, and the random-order model. For the first setting, we devised a meta-algorithm which takes any offline algorithm $\mathcal{A}_\alpha$ with an AAR of α (where α can be any constant ≥ 1), and produces an online algorithm with an ECR of $(\alpha + \varepsilon)$. This shows that online BP under the i.i.d. model and offline BP are almost equivalent. Using any AFPTAS as $\mathcal{A}_\alpha$ results in an online algorithm with an ECR of $(1 + \varepsilon)$ for any constant $\varepsilon > 0$. An interesting question of theoretical importance is to find whether achieving an ECR of 1 is possible or not. Another related open question is if we can settle online BP under the random-order model as well.

Then, we studied the analysis of the well-known BF algorithm under the random-order model. First, we proved that the ARR of BF is equal to one if all the item sizes are greater than $1/3$. Then, we improved the analysis of the BF from 1.5 to ≈ 1.49107 , for the special case when the item sizes are in the range $(1/4, 1/2]$, which corresponds to the 3-partition case. Extending the techniques for the 3-partition case, Hebbbar et al. [26] have managed to break the barrier of $3/2$ on the upper bound of ARR of BF to achieve an upper bound of $3/2 - \varepsilon$ for some $\varepsilon > 10^{-10}$. However, the conjecture that the ARR of BF lies close to 1.15 by Kenyon [35] is wide open.

References

- [1] Susanne Albers, Arindam Khan, and Leon Ladewig. 2021. Best fit bin packing with random order revisited. *Algorithmica* 83, 9 (2021), 2833–2858.
- [2] Susanne Albers, Arindam Khan, and Leon Ladewig. 2021. Improved online algorithms for knapsack and GAP in the random order model. *Algorithmica* 83, 6 (2021), 1750–1785.
- [3] Brenda S. Baker and Edward G. Coffman, Jr. 1981. A tight asymptotic bound for next-fit-decreasing bin-packing. *SIAM Journal on Algebraic Discrete Methods* 2, 2 (1981), 147–152.
- [4] János Balogh, József Békési, György Dósa, Leah Epstein, and Asaf Levin. 2018. A new and improved algorithm for online bin packing. In *ESA*, Vol. 112, 5:1 – 5:14.
- [5] János Balogh, József Békési, György Dósa, Leah Epstein, and Asaf Levin. 2021. A new lower bound for classic online bin packing. *Algorithmica* 83, 7 (2021), 2047–2062.
- [6] Rémi Bardenet and Odalric-Ambrym Maillard. 2015. Concentration inequalities for sampling without replacement. *Bernoulli* 21, 3 (2015), 1361–1385.
- [7] József Békési, Gábor Galambos, and Hans Kellerer. 2000. A $5/4$ linear time bin packing algorithm. *Journal of Computer and System Sciences* 60, 1 (2000), 145–160.
- [8] Jon Louis Bentley, David S. Johnson, Frank Thomson Leighton, Catherine C. McGeoch, and Lyle A. McGeoch. 1984. Some unexpected expected behavior results for bin packing. In *STOC*, 279–288.
- [9] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press.
- [10] Carsten Oliver Fischer. 2019. New Results on the Probabilistic Analysis of Online Bin Packing and Its Variants. Ph.D. thesis. Rheinische Friedrich-Wilhelms-Universität Bonn.
- [11] Edward G. Coffman, Jr., János Csirik, Gábor Galambos, Silvano Martello, and Daniele Vigo. 2013. Bin packing approximation algorithms: Survey and classification. In *Handbook of Combinatorial Optimization*. Panos M. Pardalos, Ding-Zhu Du, and Ronald L. Graham (Eds.), Springer, New York, 455–531.
- [12] Edward G. Coffman, Jr., David S. Johnson, George S. Lueker, and Peter W. Shor. 1993. Probabilistic analysis of packing and related partitioning problems. *Statistical Science* 8, 1 (1993), 40–47.
- [13] Edward G. Coffman, Jr., David S. Johnson, Peter W. Shor, and Richard R. Weber. 1997. Bin packing with discrete item sizes, part II: Tight bounds on first fit. *Random Structures & Algorithms* 10, 1–2 (1997), 69–101.
- [14] Edward G. Coffman, Jr., Kimming So, Micha Hofri, and A. C. Yao. 1980. A stochastic model of bin-packing. *Information and Control* 44, 2 (1980), 105–115.
- [15] Janos Csirik, David S. Johnson, Claire Kenyon, James B. Orlin, Peter W. Shor, and Richard R. Weber. 2006. On the sum-of-squares algorithm for bin packing. *Journal of the ACM* 53, 1 (2006), 1–65.
- [16] W. Fernandez de la Vega and George S. Lueker. 1981. Bin packing can be solved within $1+\varepsilon$ in linear time. *Combinatorica* 1, 4 (1981), 349–355.
- [17] Brian C. Dean, Michel X. Goemans, and Jan Vondrák. 2008. Approximating the stochastic knapsack problem: The benefit of adaptivity. *Mathematics of Operations Research* 33, 4 (2008), 945–964.
- [18] Friedrich Eisenbrand, Dömötör Pálvölgyi, and Thomas Rothvoß. 2013. Bin packing via discrepancy of permutations. *ACM Transactions on Algorithms* 9, 3 (2013), 1–15.

- [19] Jon Feldman, Aranyak Mehta, Vahab Mirrokni, and Shan Muthukrishnan. 2009. Online stochastic matching: Beating $1-1/e$. In *FOCS*, 117–126.
- [20] Thomas S. Ferguson. 1989. Who solved the secretary problem? *Statistical Science* 4, 3 (1989), 282–289.
- [21] Carsten Fischer and Heiko Röglin. 2016. Probabilistic analysis of the dual next-fit algorithm for bin covering. In *LATIN*, 469–482.
- [22] Carsten Fischer and Heiko Röglin. 2018. Probabilistic analysis of online (class-constrained) bin packing and bin covering. In *LATIN*, Vol. 10807. Springer, 461–474.
- [23] Anupam Gupta, Ravishankar Krishnaswamy, and R. Ravi. 2012. Online and stochastic survivable network design. *SIAM Journal on Computing* 41, 6 (2012), 1649–1672.
- [24] Anupam Gupta, Amit Kumar, Viswanath Nagarajan, and Xiangkun Shen. 2021. Stochastic load balancing on unrelated machines. *Mathematics of Operations Research* 46, 1 (2021), 115–133.
- [25] Anupam Gupta and Sahil Singla. 2020. Random-order models. In *Beyond the Worst-Case Analysis of Algorithms*. Tim Roughgarden (Ed.), Cambridge University Press, 234–258.
- [26] Anish Hebbar, Arindam Khan, and K. V. N. Sreenivas. 2024. Bin packing under random-order: Breaking the barrier of $3/2$. In *SODA*, 4177–4219.
- [27] Rebecca Hoberg and Thomas Rothvoss. 2017. A logarithmic additive integrality gap for bin packing. In *SODA*, 2616–2625.
- [28] David S. Johnson. 1973. Near-Optimal Bin Packing Algorithms. Ph.D. thesis. Massachusetts Institute of Technology.
- [29] David S. Johnson. 1974. Fast algorithms for bin packing. *Journal of Computer and System Sciences* 8, 3 (1974), 272–314.
- [30] David S. Johnson, Alan Demers, Jeffrey D. Ullman, Michael R. Garey, and Ronald L. Graham. 1974. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing* 3, 4 (1974), 299–325.
- [31] David S. Johnson and Michael R. Garey. 1985. A $71/60$ theorem for bin packing. *Journal of Complexity* 1, 1 (1985), 65–106.
- [32] Edward G. Coffman, Jr., János Csirik, Lajos Rónyai, and Ambrus Zsbán. 2008. Random-order bin packing. *Discrete Applied Mathematics* 156, 14 (2008), 2810–2816.
- [33] Narendra Karmarkar and Richard M. Karp. 1982. An efficient approximation scheme for the one-dimensional bin-packing problem. In *FOCS*, 312–320.
- [34] Richard M. Karp, Michael Luby, and A. Marchetti-Spaccamela. 1984. A probabilistic analysis of multidimensional bin packing problems. In *STOC*, 289–298.
- [35] Claire Kenyon. 1996. Best-fit bin-packing with random order. In *SODA*, 359–364.
- [36] Chan C. Lee and Der-Tsai Lee. 1985. A simple on-line bin-packing algorithm. *Journal of ACM* 32, 3 (1985), 562–572.
- [37] Chan C. Lee and Der-Tsai Lee. 1987. *Robust On-Line Bin Packing Algorithms*. Technical Report. Northwestern University.
- [38] Tom Leighton and Peter Shor. 1989. Tight bounds for minimax grid matching with applications to the average case analysis of algorithms. *Combinatorica* 9, 2 (1989), 161–187.
- [39] Mohammad Mahdian and Qiqi Yan. 2011. Online bipartite matching with random arrivals: An approach based on strongly factor-revealing LPs. In *STOC*, 597–606.
- [40] Frank D. Murgolo. 1988. Anomalous behavior in bin packing algorithms. *Discrete Applied Mathematics* 21, 3 (1988), 229–243.
- [41] Alantha Newman, Ofer Neiman, and Aleksandar Nikolov. 2012. Beck’s three permutations conjecture: A counterexample and some consequences. In *FOCS*, 253–262.
- [42] Prakash V. Ramanan. 1989. Average-case analysis of the smart next fit algorithm. *Information Processing Letters* 31, 5 (1989), 221–225.
- [43] Wansoo T. Rhee. 1994. Inequalities for bin packing-III. *Optimization* 29, 4 (1994), 381–385.
- [44] Wansoo T. Rhee and Michel Talagrand. 1987. Martingale inequalities and NP-complete problems. *Mathematics of Operations Research* 12, 1 (1987), 177–181.
- [45] Wansoo T. Rhee and Michel Talagrand. 1988. Exact bounds for the stochastic upward matching problem. *Transactions of the American Mathematical Society* 307, 1 (1988), 109–125.
- [46] Wansoo T. Rhee and Michel Talagrand. 1993. On-line bin packing of items of random sizes, II. *SIAM Journal on Computing* 22, 6 (1993), 1251–1256.
- [47] Peter W. Shor. 1986. The average-case analysis of some on-line algorithms for bin packing. *Combinatorica* 6, 2 (1986), 179–200.
- [48] Joel Spencer. 1994. *Ten Lectures on the Probabilistic Method*. SIAM.

Appendix

A Greedy Algorithm for Maximum Upright Matching

Here we show that Algorithm 1 correctly computes a maximum upright matching.

LEMMA A.1. *For a given set P^+ of m plus points and a set P^- of m minus points, Algorithm 1 outputs a maximum upright matching.*

PROOF. For the purpose of the proof, we denote a valid upright matching of a given instance by a set of pairs (p^+, p^-) where p^+ is a plus point and p^- is a minus point. Let Opt denote an optimal (maximum) matching and let Alg denote the matching output by Algorithm 1 and suppose that $\text{Alg} \neq \text{Opt}$. We will show that Alg is optimal via an exchange argument. In more detail, we show that we can modify Opt to obtain another optimal matching Opt' which is “closer” to Alg . By repeating this modification process, we can reach Alg in a finite number of steps, thus showing that Alg is optimal.

Without loss of generality, let us assume that all the points in the input instance have distinct x -coordinates and distinct y -coordinates. Further, for a point p , let $x(p)$ denote its x -coordinate and let $y(p)$ denote its y -coordinate. Define

$$\pi^+ = \arg \min_{p^+ : (p^+, p^-) \in \text{Alg} \Delta \text{Opt}} x(p^+),$$

where $\text{Alg} \Delta \text{Opt}$ denotes the symmetric difference between Alg and Opt . In words, π^+ is the first plus point processed by Algorithm 1 that made Opt and Alg to differ. Now, we are ready to modify Opt and obtain Opt' without decreasing the number of matched pairs. We do so by considering three cases.

Case 1: π^+ is matched in Alg but not in Opt .

Say π^+ is matched to p^- in Alg . By the optimality of Opt , the point p^- must be matched to some p^+ in Opt . We can then obtain the new optimal matching $\text{Opt}' = \text{Opt} \cup \{(\pi^+, p^-)\} \setminus \{(p^+, p^-)\}$.

Case 2: π^+ is matched in Opt but not in Alg .

We show that this case cannot arise at all. Say π^+ is matched to p^- in Opt . In Alg , π^+ is not matched, which means that p^- is matched to some $p^+ \neq \pi^+$ in Alg . By the definition of π^+ , it must be the case that p^+ is processed after π^+ by Alg . But then p^- must have been unmatched by the time π^+ is processed and Algorithm 1 would not have left π^+ unmatched.

Case 3: π^+ is matched in both Opt and Alg .

Say π^+ is matched with p_1^- in Alg and with p_2^- in Opt . If p_1^- is unmatched in Opt , then we can define a new optimal matching $\text{Opt}' = \text{Opt} \cup \{(\pi^+, p_1^-)\} \setminus \{(\pi^+, p_2^-)\}$. Now, suppose p_1^- is matched to p^+ in Opt . Then we claim that $y(p_1^-) \geq y(p_2^-)$. This is because at the time π^+ was considered by Algorithm 1, both p_1^- and p_2^- must have been unmatched. (Otherwise, it violates the definition of π^+ .) Since Algorithm 1 preferred p_1^- over p_2^- , it must be the case that $y(p_1^-) \geq y(p_2^-)$. Hence (p^+, p_2^-) is a valid matching pair. Thus we can obtain the new optimal matching $\text{Opt}' = \text{Opt} \cup \{(\pi^+, p_1^-), (p^+, p_2^-)\} \setminus \{(\pi^+, p_2^-), (p^+, p_1^-)\}$.

That ends the construction of Opt' . To conclude the proof, we define

$$x^+(\mathcal{M}_1, \mathcal{M}_2) = \min_{(p^+, p^-) \in \mathcal{M}_1 \Delta \mathcal{M}_2} x(p^+)$$

for any two matchings $\mathcal{M}_1, \mathcal{M}_2$, and observe that $x^+(\text{Opt}', \text{Alg})$ is strictly greater than $x^+(\text{Opt}, \text{Alg})$. $\square$

B Optimal Packing Size Is Almost Proportional to Input Length

In this section, we show 1. First, we use the following lemma, which is almost similar to Theorem 2.1 in [46].

LEMMA B.1. *Let J be an arbitrary set of q items and let S be a set of p items (with $p \leq q$) sampled uniformly randomly without replacement from J . Then there exist constants K', a' such that*

$$\mathbb{P} \left[\text{Opt}(S) \geq \frac{p}{q} \text{Opt}(J) + K' \sqrt{q} (\log q)^{3/4} \right] \leq \exp \left(-a' (\log q)^{3/2} \right).$$

Let $I_{\text{samp}}(t)$ denote a set of t items sampled uniformly randomly *without* replacement from I . Then, using the above lemma, we obtain that

$$\mathbb{P} \left[\text{Opt}(I_{\text{samp}}(t)) \geq \frac{t}{n} \text{Opt}(I) + K' \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a' (\log n)^{3/2} \right). \quad (\text{B.1})$$

Then, we observe that the lists $I(1, t)$ and $I_{\text{samp}}(t)$ have the same joint distribution. This implies that both the quantities $\text{Opt}(I(1, t))$ and $\text{Opt}(I_{\text{samp}}(t))$ are roughly the same.

PROPOSITION B.2. *There exist constants K'', a'' such that*

$$\mathbb{P} \left[\text{Opt}(I(1, t)) - \text{Opt}(I_{\text{samp}}(t)) \geq K'' \sqrt{n} (\log n)^{3/4} \right] \leq \exp \left(-a'' (\log n)^{3/2} \right).$$

To conclude, we observe that combining Equation (B.1) and the above proposition gives us the final claim. We are left with proving Lemma B.1 and Proposition B.2.

PROOF OF LEMMA B.1. The proof is long, but is essentially the same as that of Theorem 2.1 in [46]. The only difference is the use of concentration inequalities for sampling without replacement, instead of sampling with replacement.

Let $(Y_1, Y_2, \dots, Y_q)$ denote the set J and let $s(1), s(2), \dots, s(p)$ denote p indices that are sampled uniformly randomly from $[q]$ without replacement. Hence $(Y_{s(1)}, Y_{s(2)}, \dots, Y_{s(p)})$ denotes the set S . We first compute an optimal packing of J . We refer to this packing as *model packing* and denote it by $\mathcal{P}$. Without loss of generality, we assume that in each bin of $\mathcal{P}$, the items are ordered in non-increasing order of weights. For an item $j \in J$, we define $\text{rank}(j)$ as the position of j in the bin in which j is placed in $\mathcal{P}$.

Now, we show how to pack S . For every rank $r \in \{2, 3, \dots, p\}$, we maintain a set $V(r)$ of pairs of the form (B, x) . Initially, all these sets are empty. We process $Y_{s(1)}, Y_{s(2)}, \dots, Y_{s(p)}$ in that order. Some of these items will be tagged as *overflow items*. All the overflow items will be packed at once using NF. Assume we are at an intermediate stage where we have packed $Y_{s(1)}, \dots, Y_{s(i-1)}$ and are about to pack $Y_{s(i)}$.

- (1) If $\text{rank}(Y_{s(i)}) = 1$, then we open a new bin B and pack $Y_{s(i)}$ there. In addition, for each rank $r \in \{2, 3, \dots, m_i\}$ (where m_i is the number of items in the bin in $\mathcal{P}$ containing $Y_{s(i)}$), we add the pair (B, x) to the list $V(r)$, where x is the weight of the r th largest item in the bin in $\mathcal{P}$ containing $Y_{s(i)}$.
- (2) If $r := \text{rank}(Y_{s(i)}) \in \{2, \dots, p\}$, we look at all the pairs in $V(r)$ and identify the pair (B, x) , if one exists, such that x is least possible but at least the weight of $Y_{s(i)}$. We then pack $Y_{s(i)}$ in B and remove the pair (B, x) from the list $V(r)$. If no such pair in $V(r)$ exists, then we tag $Y_{s(i)}$ as an overflow item.
- (3) If $r := \text{rank}(Y_{s(i)}) > p$, we tag $Y_{s(i)}$ as an overflow item.

In this way, we pack all the items in S . We now analyze the number of bins used in this process. First, observe that, ignoring the overflow items, we open a new bin only when an item of rank 1 appears. Let N_{main} denote this number of bins. Since S is obtained by sampling p items from J without replacement, by using Hoeffding's inequality for sampling without replacement (see [6]), we can bound N_{main} .

We state the Hoeffding's inequality now.

PROPOSITION B.3 (Hoeffding's Inequality). *Let $(Z_1, Z_2, \dots, Z_M)$ be a list of M arbitrary real numbers in $[0, 1]$ and let $\mu = \frac{1}{M} \sum_j Z_j$ be the mean of this list. Let $(z_1, z_2, \dots, z_m)$ be a list of m numbers sampled from the original list uniformly at random without replacement. Then for any $\delta > 0$,*

$$\mathbb{P} \left[\sum_{i \in [m]} z_i - \mu m > \delta \right] \leq \exp \left(-\frac{2\delta^2}{m} \right).$$

To bound N_{main} , we use the Hoeffding's inequality by setting

- (1) $M = q$ and $m = p$.
- (2) $Z_j = 1$ if $\text{rank}(Y_j) = 1$ and $Z_j = 0$ otherwise.

Then, we obtain that $\mu = 1/q \sum_j Z_j = \text{Opt}(J)/q$ and $\sum_i z_i = N_{\text{main}}$. Hence, we obtain

$$\mathbb{P} \left[N_{\text{main}} \geq \frac{p}{q} \text{Opt}(J) + \sqrt{p}(\log q)^{3/4} \right] \leq \exp(-2(\log q)^{3/2}).$$

We are left with bounding the number of bins to pack the overflow items. We denote the total weight by W_{of} and the number of bins used to pack them by N_{of} . Since we use NF to pack the overflow items, we have $N_{\text{of}} \leq 2W_{\text{of}} + 1$. Hence, we only need to bound W_{of} . Note that any item of rank r can have weight at most $1/r$. Hence the total weight of items of rank more than p (those considered in step 3 of our procedure) is at most 1. This leaves us with the task of bounding the total weight of overflow items of rank in $\{2, 3, \dots, p\}$ (step 2 in our procedure).

For a rank $r \in \{2, \dots, p\}$, let n_r denote the number of rank r overflow items. We can bound n_r by reducing it to a variant of upright matching. For each $r \in \{2, \dots, p\}$, we gradually construct an instance of upright matching $\mathcal{M}_r$ as we process S as follows. Suppose we are processing $Y_{s(i)}$.

- (1) Say $\text{rank}(Y_{s(i)}) = 1$ and there are at least r items in the bin in $\mathcal{P}$ containing $Y_{s(i)}$. Then we create a minus point (i, w) where w is the weight of the rank r item in $\mathcal{P}$ containing $Y_{s(i)}$.
- (2) Say $\text{rank}(Y_{s(i)}) = r$. Then we create a plus point (i, w) where w is the weight of $Y_{s(i)}$.
- (3) We do nothing in rest of the cases.

Then, it can be seen that the maximum matching procedure of Algorithm 1 on $\mathcal{M}_r$ exactly corresponds to step 2 of our packing procedure.⁵ Hence the number of unmatched plus points in $\mathcal{M}_r$ exactly corresponds to the number of overflow items of rank r .

For the sake of simplicity, and to overcome certain technical details of upright matching, we assume that instead of stopping after sampling p items from J , we continue till all the q items of J are sampled. This will only increase the number of overflow items.

Let us fix a rank $r \in \{2, \dots, p\}$. Let q_r denote the total number of items of rank r in J . From the results of [10] (see MATCHING VARIANT M2 in the reference), the number of unmatched plus points in $\mathcal{M}_r$ can be bounded as $n_r \leq K\sqrt{q_r}(\log q_r)^{3/4}$ with probability at least $1 - \exp(-a(\log q_r)^{3/2})$ for some constant $a > 3$. Distinguishing between the cases $q_r \leq q^{1/3}$ and $q_r \geq q^{1/3}$, we obtain that

$$n_r \leq q^{1/3} + K\sqrt{q_r}(\log q_r)^{3/4},$$

with probability at least $1 - \exp(-\frac{a}{3}(\log q)^{3/2})$. Now applying a union bound, we obtain that for all $r \in \{2, \dots, p\}$, we have

$$n_r \leq q^{1/3} + K\sqrt{q_r}(\log q)^{3/4}$$

with probability at least $1 - q \exp(-\frac{a}{3}(\log q)^{3/2})$ which is at least $1 - \exp(-(\frac{a}{3} - 1)(\log q)^{3/2})$.

⁵Except for reflection about x-axis.

Hence we obtain that the total number of overflow items is at most

$$\sum_{r=2}^p \frac{n_r}{r} \leq q^{1/3} \log q + K(\log q)^{3/4} \sum_{r=2}^p \frac{\sqrt{q_r}}{r}.$$

Applying Cauchy–Schwartz inequality and using the fact that $\sum_{r=2}^p q_r \leq q$, we obtain that

$$\sum_{r=2}^p \frac{\sqrt{q_r}}{r} \leq \sqrt{\sum_{r=2}^p q_r} \sqrt{\sum_{r=2}^p \frac{1}{r^2}} \leq K\sqrt{q},$$

thus ending the proof. $\square$

PROOF OF PROPOSITION B.2. Let $X_1, X_2, \dots, X_n$ be the items in the input set I . Let $s(1), s(2), \dots, s(t)$ denote t indices from $[n]$ sampled uniformly randomly without replacement. Therefore, $I(1, t) = (X_1, X_2, \dots, X_t)$ and $I_{\text{samp}}(t) = (X_{s(1)}, X_{s(2)}, \dots, X_{s(t)})$. We now show that $I(1, t)$ and $I_{\text{samp}}(t)$ have the same joint distribution by showing that the joint cumulative distribution functions are equal. Let $x_1, x_2, \dots, x_t$ be some arbitrary reals in $[0, 1]$. Then

$$\begin{aligned} & \mathbb{P} \left[(X_{s(1)} \leq x_1) \wedge (X_{s(2)} \leq x_2) \wedge \dots \wedge (X_{s(t)} \leq x_t) \right] \\ &= \sum_{\substack{s_1, s_2, \dots, s_t \in [n] \\ \text{all different}}} \mathbb{P} \left[\bigwedge_{i \in [t]} X_{s(i)} \leq x_i \mid \bigwedge_{i \in [t]} s(i) = s_i \right] \mathbb{P} \left[\bigwedge_{i \in [t]} s(i) = s_i \right] \\ &= \sum_{\substack{s_1, s_2, \dots, s_t \in [n] \\ \text{all different}}} \mathbb{P} \left[\bigwedge_{i \in [t]} X_{s_i} \leq x_i \right] \mathbb{P} \left[\bigwedge_{i \in [t]} s(i) = s_i \right] \\ &= \sum_{\substack{s_1, s_2, \dots, s_t \in [n] \\ \text{all different}}} \mathbb{P} \left[\bigwedge_{i \in [t]} X_i \leq x_i \right] \mathbb{P} \left[\bigwedge_{i \in [t]} s(i) = s_i \right] \\ &= \mathbb{P} \left[\bigwedge_{i \in [t]} X_i \leq x_i \right] \sum_{\substack{s_1, s_2, \dots, s_t \in [n] \\ \text{all different}}} \mathbb{P} \left[\bigwedge_{i \in [t]} s(i) = s_i \right] \\ &= \mathbb{P} \left[(X_1 \leq x_1) \wedge (X_1 \leq x_2) \wedge \dots \wedge (X_t \leq x_t) \right]. \end{aligned}$$

Since $I(1, t)$ and $I_{\text{samp}}(t)$ have the same joint distribution, we obtain that

$$\mathbb{E} [\text{Opt}(I(1, t))] = \mathbb{E} [\text{Opt}(I_{\text{samp}}(t))] =: \Gamma.$$

Further, Theorem 2 from [44] tells us that if a set T of m items is sampled independently and identically from a fixed distribution, then for any $\ell \geq 0$,

$$\mathbb{P} [|\text{Opt}(T) - \mathbb{E} [\text{Opt}(T)]| > \ell] \leq 2 \exp(-\ell^2/(2m)).$$

Thus, we obtain the following two inequalities, which hold for some constants K, a :

$$\begin{aligned} \mathbb{P} \left[|\text{Opt}(I(1, t)) - \Gamma| > K\sqrt{t}(\log n)^{3/4} \right] &\leq 2 \exp(-a(\log n)^{3/2}), \\ \mathbb{P} \left[|\text{Opt}(I_{\text{samp}}(t)) - \Gamma| > K\sqrt{t}(\log n)^{3/4} \right] &\leq 2 \exp(-a(\log n)^{3/2}). \end{aligned}$$

Combining both the inequalities and observing that $t \leq n$ gives us the proposition. $\square$

Received 24 January 2024; revised 25 February 2025; accepted 3 April 2025